

MundiPets — PE5 Unidad 5

Fundamento Teórico (FT.3), Sección 4 y Sección 6

Ingeniería de Requisitos (ISR-401) — UTEQ 2026–2027 PPA

Responsable: Morales Sánchez, Gary Alejandro

Fundamento Teórico

Fundamentos de la trazabilidad y el modelado

El planteamiento seminal de Gotel y Finkelstein [1] identificó que buena parte de los problemas prácticos de trazabilidad no surgen de una técnica insuficiente para vincular artefactos, sino de una falta de acuerdo previo sobre qué tipo de trazabilidad se está exigiendo. Los autores separan el problema en dos direcciones complementarias: la trazabilidad previa a la especificación, que vincula un requisito hacia atrás con la evidencia que lo originó, ya sea una entrevista, una observación o una norma legal; y la trazabilidad posterior a la especificación, que lo vincula hacia adelante con aquello en lo que ese requisito se transforma durante el desarrollo, como un caso de uso, una clase o un caso de prueba. Esta distinción sustenta directamente la columna Fuente y la columna Estado de la traza de la matriz del punto 6.2 de este informe: la primera resuelve la pregunta de dónde nació este requisito, y la segunda, la pregunta de en qué se convirtió; sin esa separación conceptual, ambas preguntas tienden a mezclarse en una sola verificación superficial que no detecta en cuál de las dos direcciones falla realmente la cadena.

La revisión sistemática más reciente de Mucha, Kaufmann y Riehle [2], basada en el análisis de setenta y siete estudios publicados entre 1992 y 2022, retoma específicamente la mitad de ese marco correspondiente a la trazabilidad previa a la especificación, señalada por los propios autores como la menos estudiada de las dos, y documenta de forma sistemática los problemas recurrentes que enfrenta un equipo al intentar mantenerla vigente durante todo el ciclo de vida del proyecto. Entre esos problemas se encuentran precisamente las dos patologías que este informe debió diagnosticar y cerrar en la Sección 6.3: la cadena rota, cuando un artefacto de diseño, como un caso de uso o una clase, se construye sin que ningún requisito lo respalde realmente, señal de alcance no solicitado; y el requisito huérfano, cuando un requisito carece de fuente identificable o de cualquier artefacto posterior que lo realice, señal de que fue documentado sin evidencia suficiente o abandonado tras su especificación. Que estas patologías aparezcan documentadas de forma independiente en la literatura de trazabilidad, y no solo como una ocurrencia particular de este proyecto, respalda que su tratamiento en la Sección 6.3 no fue un ejercicio cosmético, sino la aplicación de un problema ya caracterizado formalmente en la disciplina.

La cadena mínima que exige la PE5, compuesta por Fuente, RF, CU, Clase, Estado, Criterio BDD y Caso de Prueba, es en consecuencia la aplicación directa de este marco al proyecto MundiPets: sus primeros dos eslabones, Fuente y RF, instancian la trazabilidad previa a la especificación de Gotel y Finkelstein, y los eslabones restantes, desde RF hasta el Caso de Prueba pasando por CU, Clase, Estado y Criterio BDD, instancian su trazabilidad posterior a la especificación. Verificar ambas direcciones en una sola matriz, en lugar de documentar solamente el origen de los requisitos o solamente su realización en los modelos de la Sección 4, es lo que permite detectar tanto las cadenas rotas como los huérfanos que describen Mucha et al., y es exactamente el criterio bajo el cual se construyeron y depuraron la lectura interpretativa de los modelos UML de la Sección 4 y la matriz final del punto 6.2.

1 Modelos UML del Sistema

Esta sección presenta los modelos vigentes de la PE3 ya corregidos, con una lectura interpretativa de la decisión de diseño que refleja cada uno, no una descripción de lo que se observa en el dibujo. El orden sigue el recorrido natural del sistema: primero los modelos generales de alcance y arquitectura (casos de uso, contexto, componentes, despliegue) y los modelos estratégicos i* (SD y SR); luego el comportamiento dinámico (máquinas de estado, secuencia, actividad); y finalmente la estructura estática (clases).

1.1 Diagrama de Casos de Uso

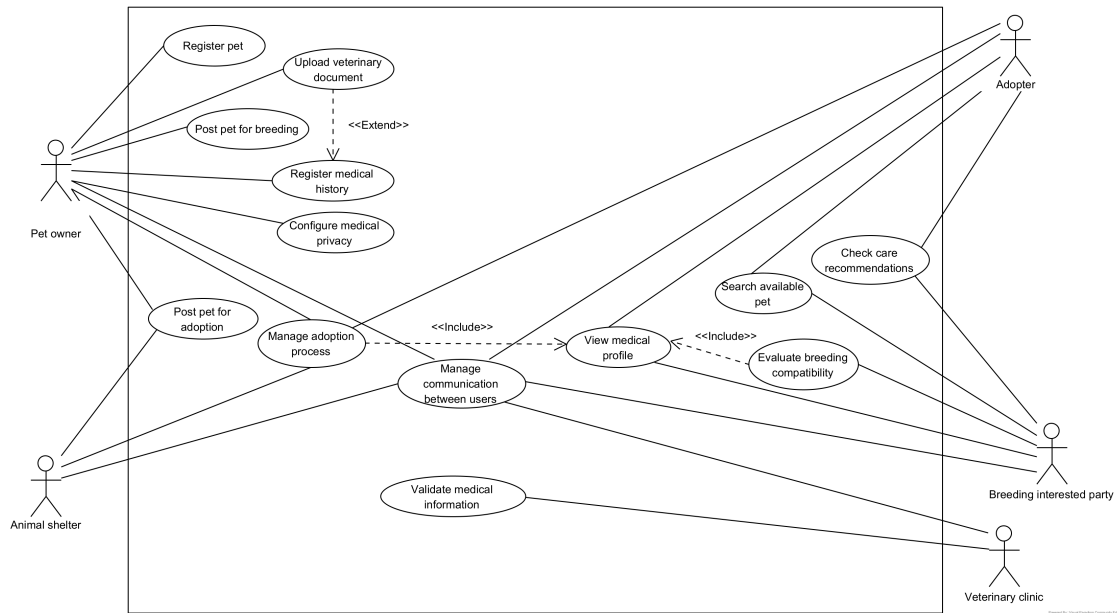


Figura 1: Diagrama de casos de uso general del sistema MundiPets.

En la Figura 1, la inclusión obligatoria de “Manage adoption process” y “Manage communication between users” hacia “View medical profile” refleja que ningún adoptante o interesado en cruce puede avanzar en una negociación sin pasar primero por el historial médico de la mascota: la transparencia sanitaria no es opcional, es un paso obligado del flujo. Por otro lado, “Upload veterinary document” extiende a “Register medical history” (no lo incluye), lo que indica que adjuntar el documento formal es un enriquecimiento posterior y no una condición para registrar el historial básico, permitiendo que el dueño registre datos médicos mínimos, aunque aún no tenga el certificado en mano.

1.2 Diagrama de Contexto

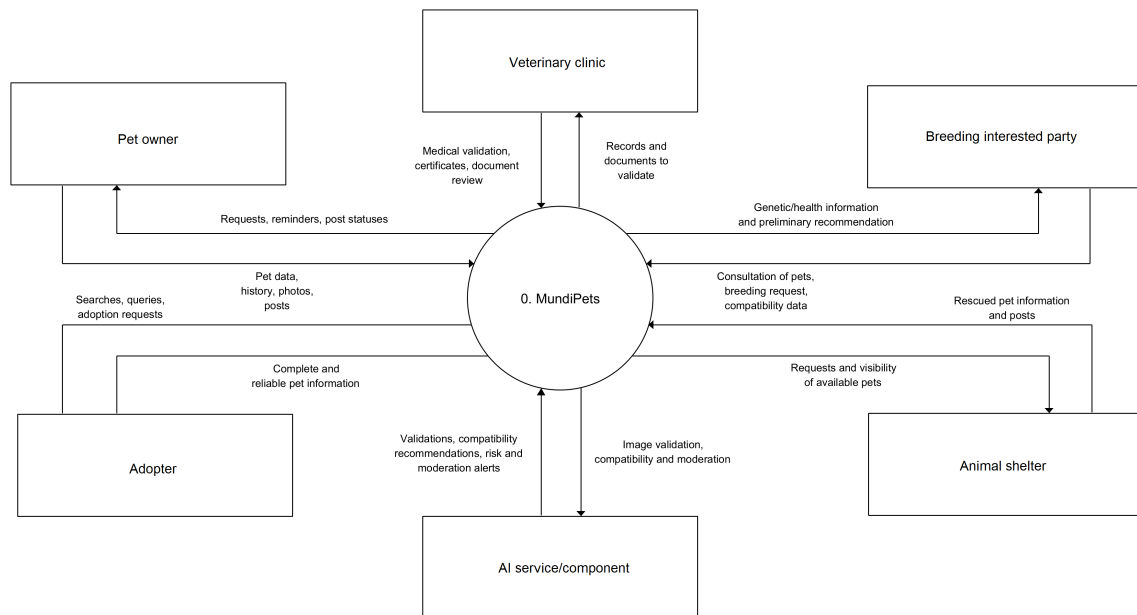


Figura 2: Diagrama de contexto del sistema MundiPets.

Como se observa en la Figura 2, la existencia de un actor externo independiente “AI service/component”, con flujo bidireccional propio hacia el núcleo “0. MundiPets” (envía moderación de imágenes y compatibilidad, recibe validaciones), refleja la decisión de tratar la inteligencia artificial como un límite del sistema desacoplado del resto de actores humanos, y no como una función interna invisible. Esto anticipa que los componentes de IA (Sección ??) tienen contratos de entrada/salida propios y auditables, no una lógica difusa mezclada con el resto del sistema.

1.3 Diagrama de Componentes

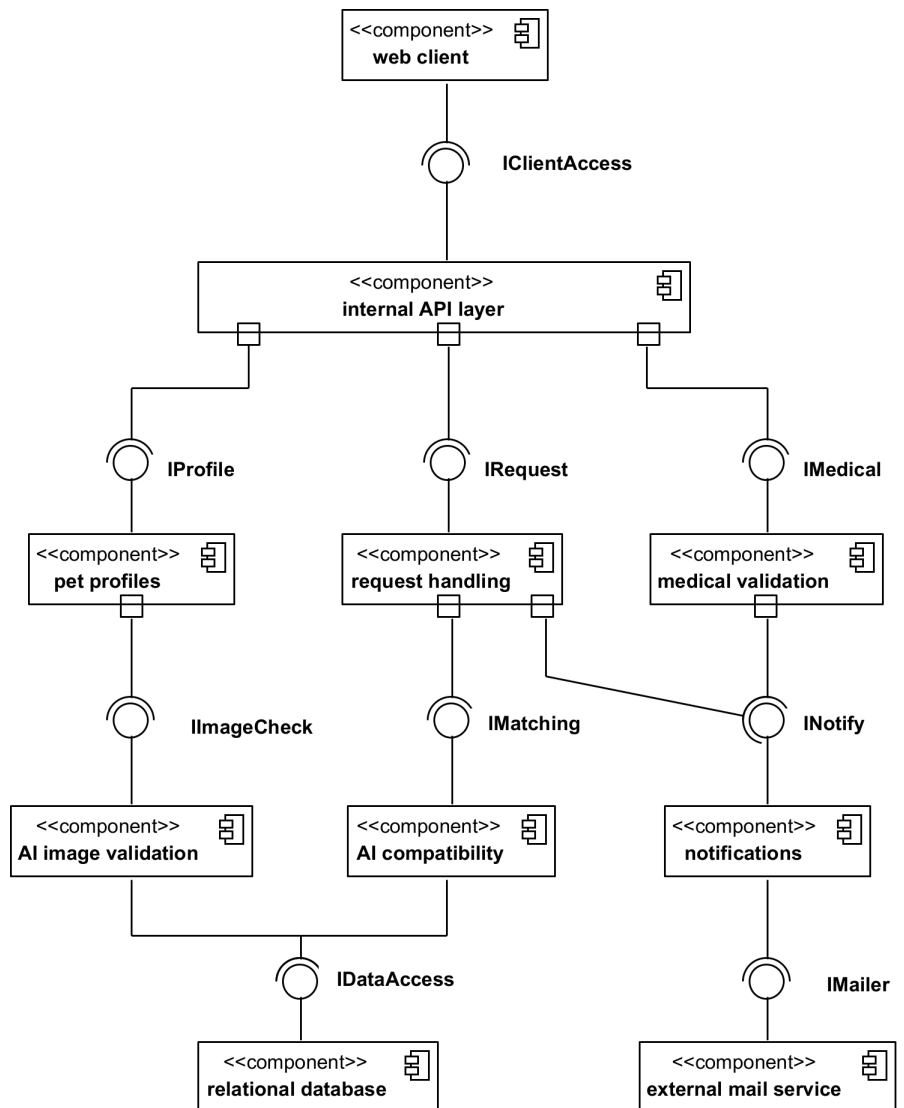


Figura 3: Diagrama de componentes del sistema MundiPets.

La Figura 3 muestra que la separación entre “AI image validation” y “AI compatibility” como componentes independientes que exponen sus propias interfaces (*IImageCheck*, *IMatching*) hacia “request handling” y “pet profiles”, en lugar de integrarse directamente en la lógica de negocio, refleja la decisión de aislar los modelos de IA del resto del sistema para poder reemplazarlos, reentrenarlos o degradarlos (*fallback*) sin tocar la capa de API. Además, que “IRequest” comparta la interfaz “INotify” con “notifications” (en vez de que cada componente notifique por su cuenta) muestra que las notificaciones se centralizaron como un servicio transversal y no como lógica duplicada en cada módulo.

1.4 Diagrama de Despliegue

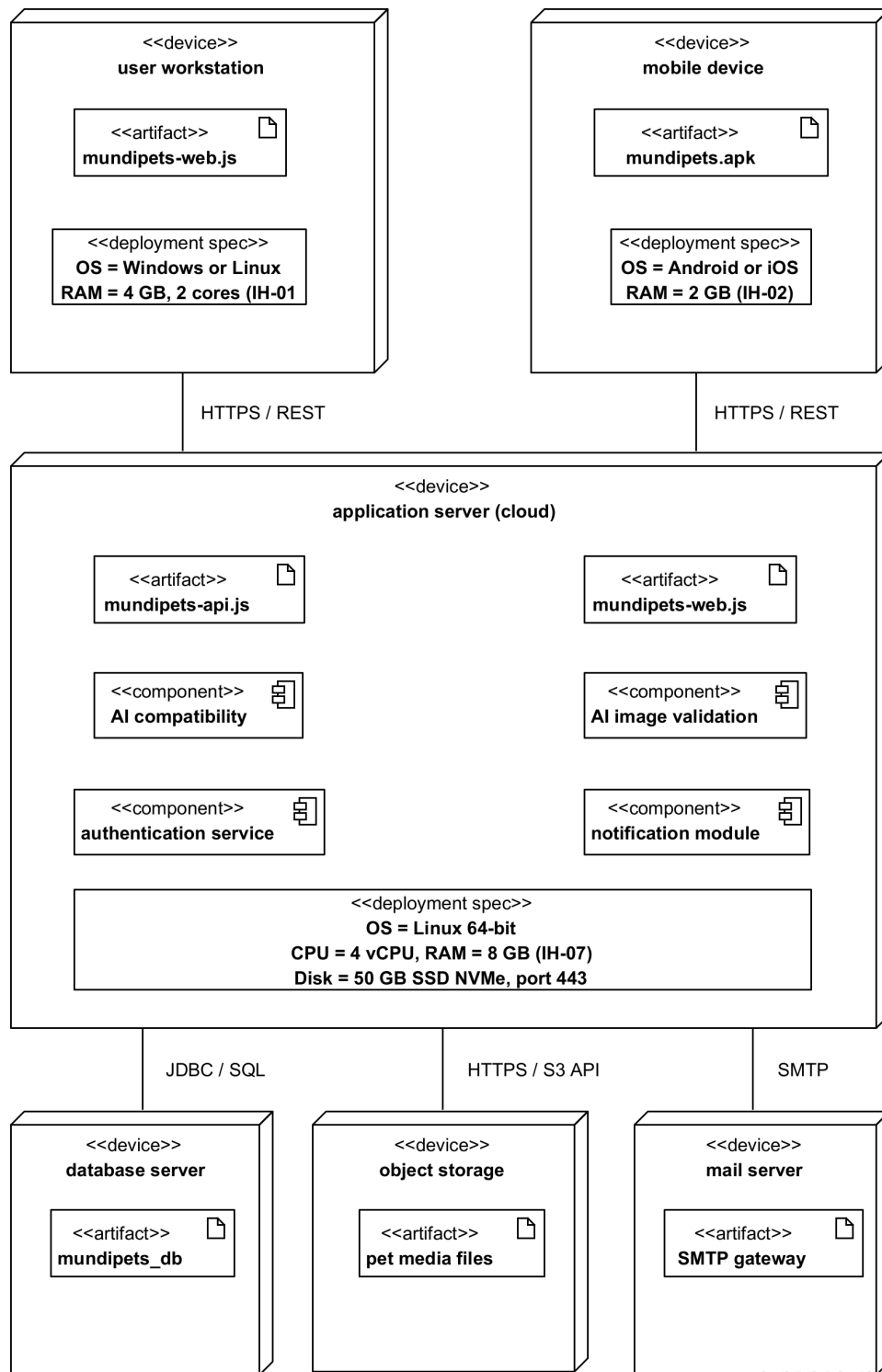


Figura 4: Diagrama de despliegue del sistema MundiPets.

Que los componentes “AI compatibility” y “AI image validation” se desplieguen dentro del mismo nodo “application server (cloud)” junto con la autenticación y las notificaciones (y no en un nodo de cómputo dedicado o servicio externo separado), como se aprecia en la Figura 4, refleja que el equipo optó por una arquitectura monolítica de despliegue para el MVP, priorizando simplicidad operativa sobre escalabilidad independiente de la IA, decisión coherente con el nivel de riesgo y volumen de datos que maneja MundiPets en esta etapa.

1.5 Diagramas Estratégicos (i*)

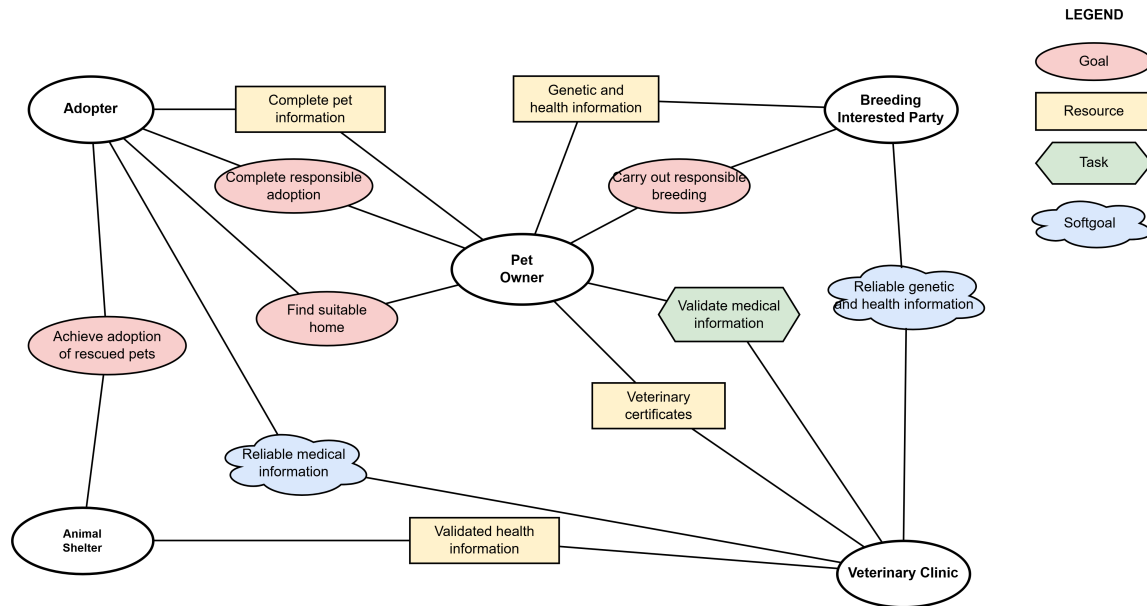


Figura 5: Diagrama de dependencia estratégica (SD) del sistema MundiPets.

En la Figura 5, que “Pet Owner” dependa de “Veterinary Clinic” para la tarea “Validate medical information” (una tarea, no un simple recurso) revela que la validación médica no es un intercambio de datos pasivo, sino un proceso activo que el dueño no puede realizar por sí mismo ni delegar a otro actor: la clínica es la única fuente legítima de verificación, lo que justifica por qué el ERS exige trazabilidad hacia un actor externo verificador y no solo un campo de texto autodeclarado.

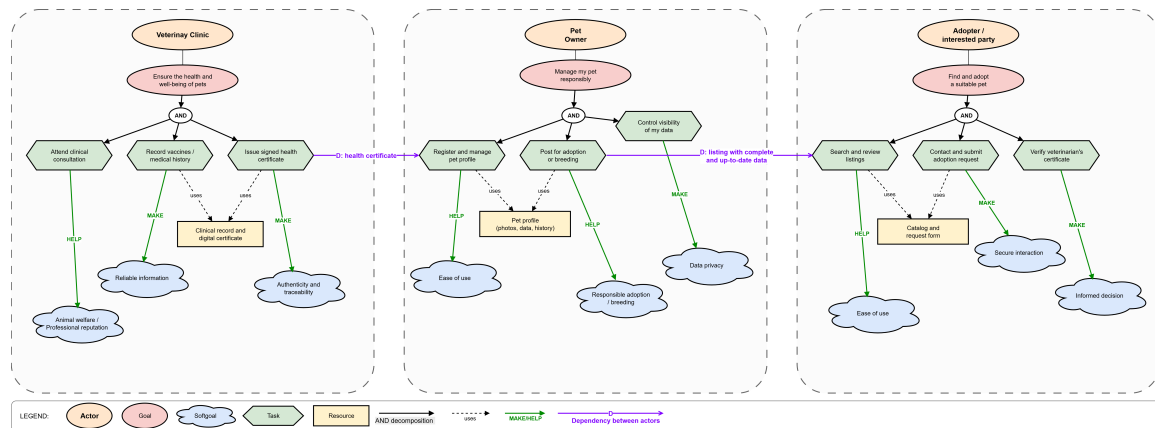


Figura 6: Diagrama de razón estratégica (SR) del sistema MundiPets.

Dentro del racional del “Pet Owner” en la Figura 6, que “Post for adoption or breeding” dependa del recurso “Pet profile (photos, data, history)” mediante *uses* (y no que ambas tareas generen su propio perfil por separado) refleja la decisión de que el perfil de la mascota es un artefacto único y compartido entre los flujos de adopción y de cruce, evitando duplicidad de datos y asegurando que cualquier corrección al perfil (por ejemplo, del historial médico) se propague automáticamente a ambos procesos.

1.6 Diagrama de Flujo de Datos (Nivel 1)

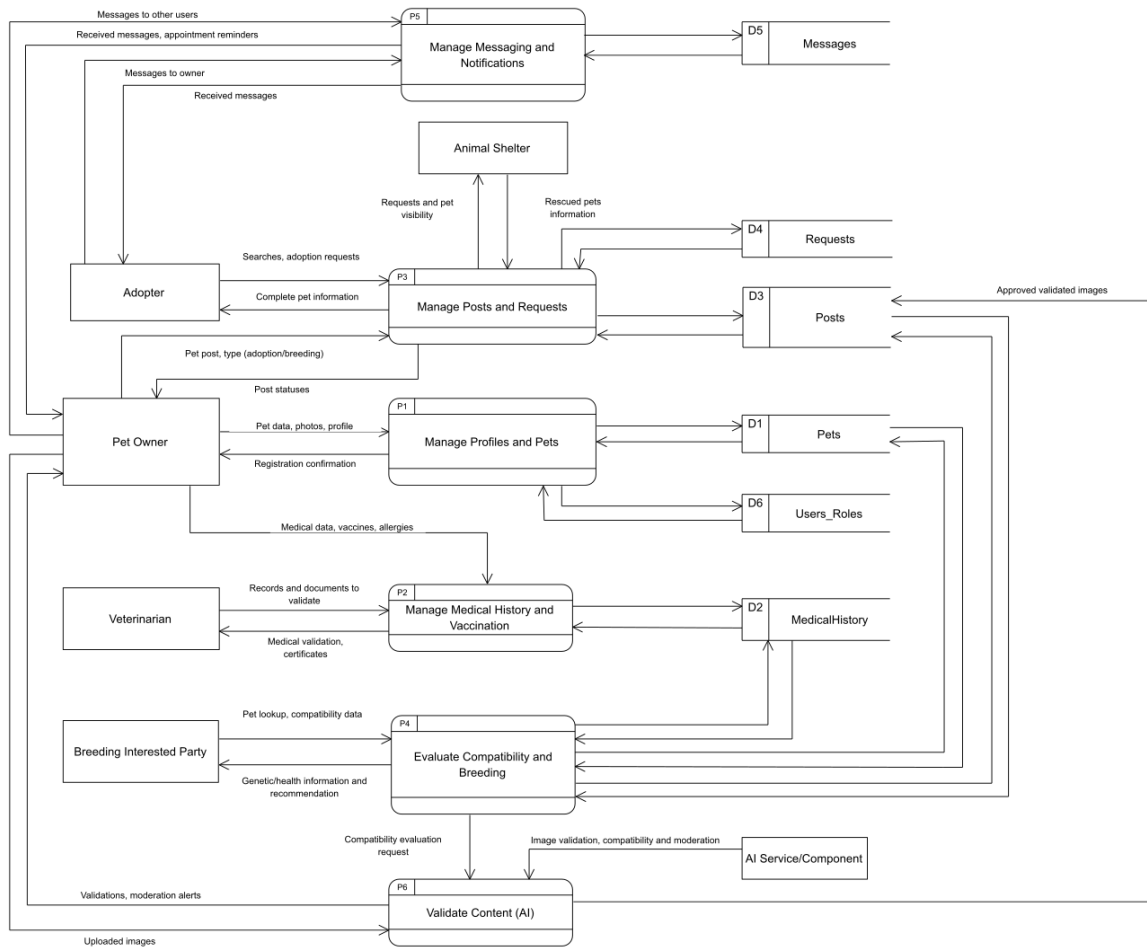


Figura 7: Diagrama de flujo de datos, Nivel 1, notación Yourdon-DeMarco.

Que el proceso único “0. MundiPets” del Diagrama de Contexto (Figura 2, que funciona como DFD Nivel 0) se descomponga en exactamente seis procesos (P1 Manage Profiles and Pets, P2 Manage Medical History and Vaccination, P3 Manage Posts and Requests, P4 Evaluate Compatibility and Breeding, P5 Manage Messaging and Notifications, P6 Validate Content) revela que el equipo agrupó la funcionalidad del sistema por dominio de datos compartido y no por actor o por caso de uso individual: P2, por ejemplo, concentra tanto el registro del historial médico (RF-02) como su validación profesional (RF-09) y la trazabilidad de vacunas (RF-14, RF-24), porque las tres operan sobre el mismo almacén D2 (MedicalHistory), aunque correspondan a actores distintos (propietario y veterinaria). Que P6 (“Validate Content”) se mantenga como proceso independiente, con su propio flujo de entrada y salida hacia el “AI Service/Component” ya visto en el Diagrama de Contexto, refuerza la misma decisión de aislamiento de la IA observada en el Diagrama de Componentes: el flujo de datos de la validación automática nunca se mezcla con el flujo de datos de negocio de los demás procesos, sino que pasa siempre por un almacén y un proceso propios.

1.7 Diagramas de Máquinas de Estado

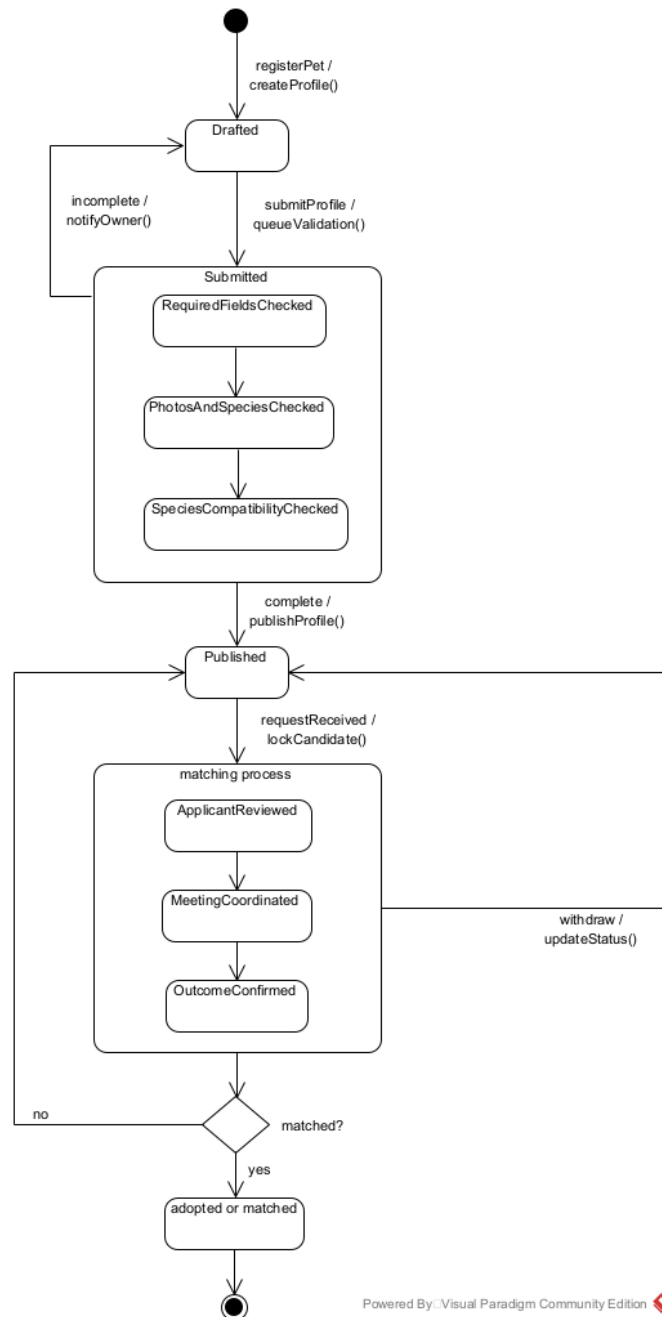


Figura 8: Diagrama de máquinas de estado – Perfil Mascota.

Que el estado “Submitted” sea un subestado compuesto con una secuencia interna obligatoria (*RequiredFieldsChecked* → *PhotosAndSpeciesChecked* → *SpeciesCompatibilityChecked*) antes de poder llegar a “Published”, como se ve en la Figura 8, revela que la publicación de una mascota nunca es un evento atómico, sino el resultado de una validación en capas donde ningún criterio puede saltarse: si falta un campo, el sistema regresa a “Drafted” y notifica al dueño en vez de dejar el perfil a medio camino. Asimismo, que “matched?” sea una decisión posterior a todo el subestado “matching process” (y no una simple bandera booleana temprana) muestra que el sistema exige agotar la revisión del solicitante, coordinación de encuentro y confirmación del resultado antes de decidir si hubo adopción o cruce exitosa, evitando cierres prematuros del caso.

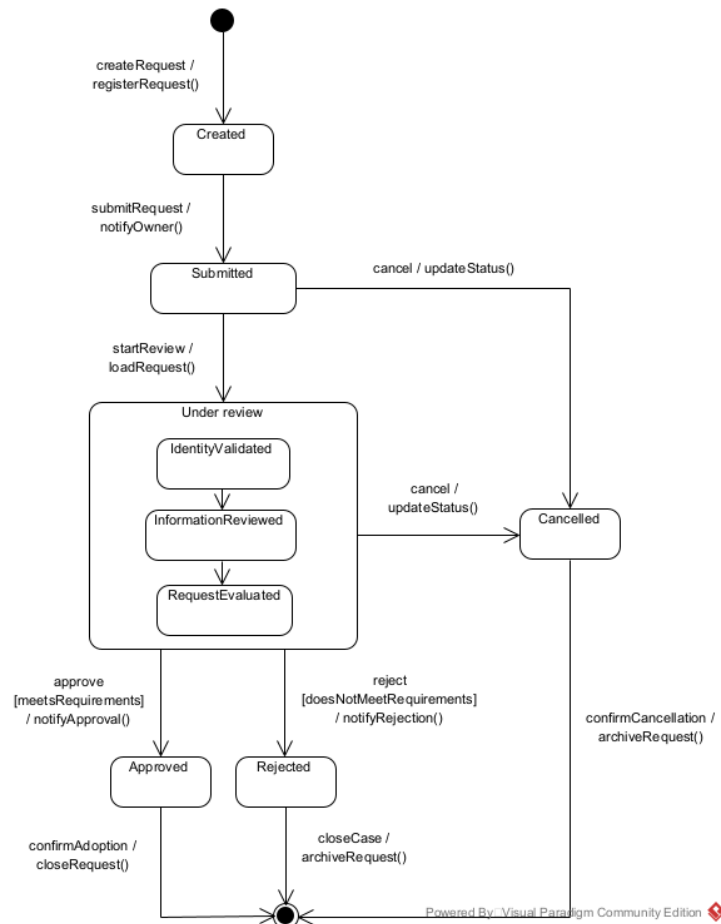


Figura 9: Diagrama de máquinas de estado – Solicitudes.

Que “cancel” esté habilitado tanto desde “Submitted” como desde dentro de “Under review” (dos transiciones distintas hacia “Cancelled”, no una sola desde el estado padre), tal como se muestra en la Figura 9, refleja que el diseño reconoce que una solicitud puede ser retirada por el interesado en cualquier punto del proceso, incluso mientras ya se está validando la identidad o evaluando la solicitud, y no únicamente antes de empezar la revisión. Además, que “Approved” y “Rejected” conduzcan a acciones finales distintas (*confirmAdoption* vs. *closeCase*, ambas con *archiveRequest()*) en lugar de una única salida genérica, indica que el cierre de una adopción concretada exige un paso adicional de confirmación que el cierre de un rechazo no necesita, correspondiendo a una responsabilidad distinta según el desenlace.

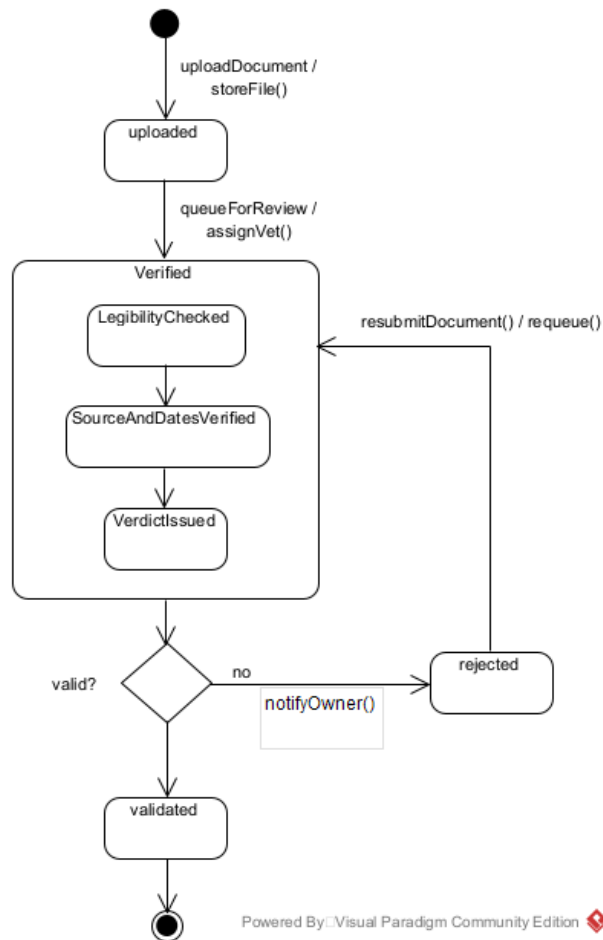


Figura 10: Diagrama de máquinas de estado – Verificación de Documentos.

Que *resubmitDocument()/requeue()* regrese exclusivamente desde “rejected” hacia el subestado compuesto “Verified” (y no hacia “uploaded”), como se aprecia en la Figura 10, revela que un documento rechazado no reinicia el ciclo completo de carga, sino que retorna directamente a la cola de revisión una vez corregido, evitando que el dueño tenga que subir el archivo desde cero. También, que la verificación interna se descomponga en tres pasos secuenciales (legibilidad → verificación de fuente y fechas → emisión de veredicto) antes de la decisión “valid?” muestra que el sistema separa expresamente la validez formal del documento (que se lea bien) de su validez de contenido (que la fuente y fechas sean correctas), permitiendo auditar en qué paso específico falló un documento rechazado.

1.8 Diagramas de Secuencia

1.8.1 Registrar mascota (CU-01)

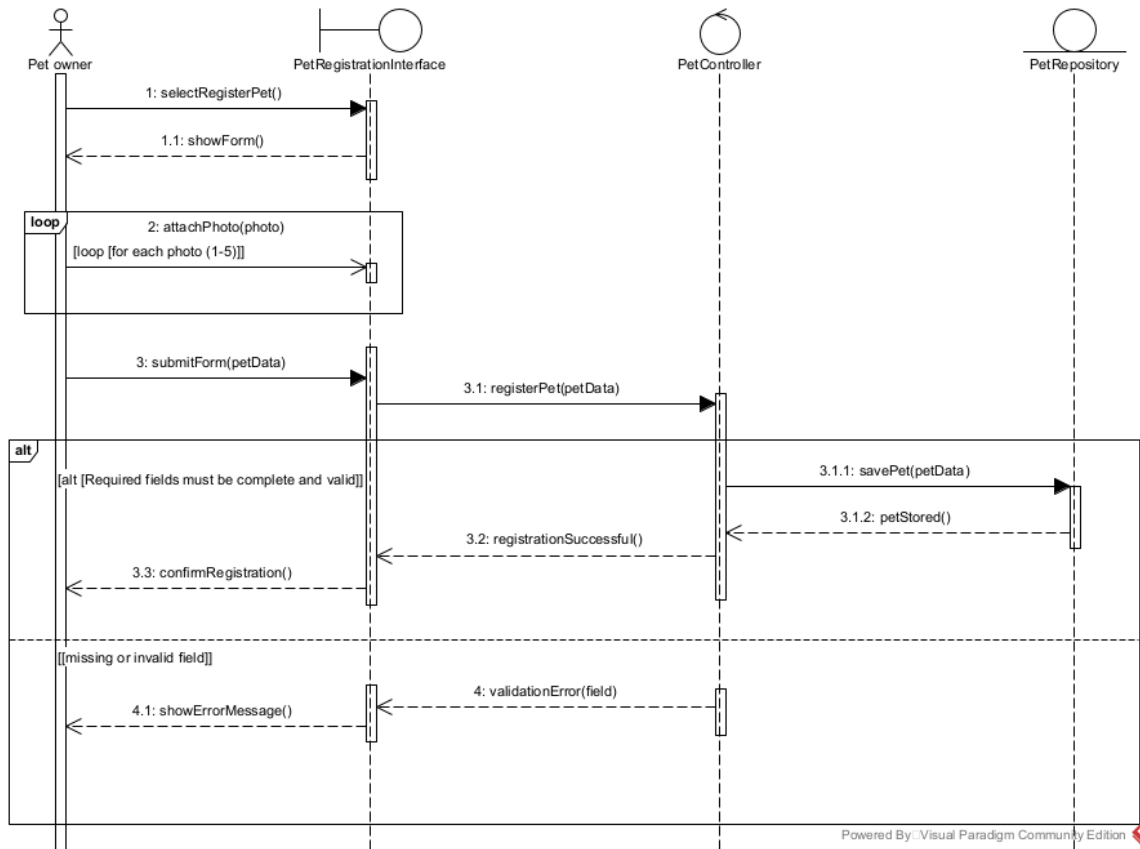


Figura 11: Diagrama de secuencia – Registrar mascota (CU-01).

En la Figura 11, que *attachPhoto(photo)* se ejecute en un *loop* previo al *submitForm(petData)* (y no como parte del mismo envío) revela que el diseño separa la carga incremental de fotos de la validación final del formulario, permitiendo que el dueño adjunte entre 1 y 5 fotos de forma progresiva antes de comprometerse a enviar el registro completo. Además, que la validación de campos ocurra en el *PetController* y no en la interfaz (el *alt* evalúa “Required fields must be complete and valid” a nivel de controlador) muestra que la lógica de negocio, no la capa de presentación, es la responsable de decidir si el registro es válido.

1.8.2 Registrar historial médico y antecedentes genéticos (CU-02)

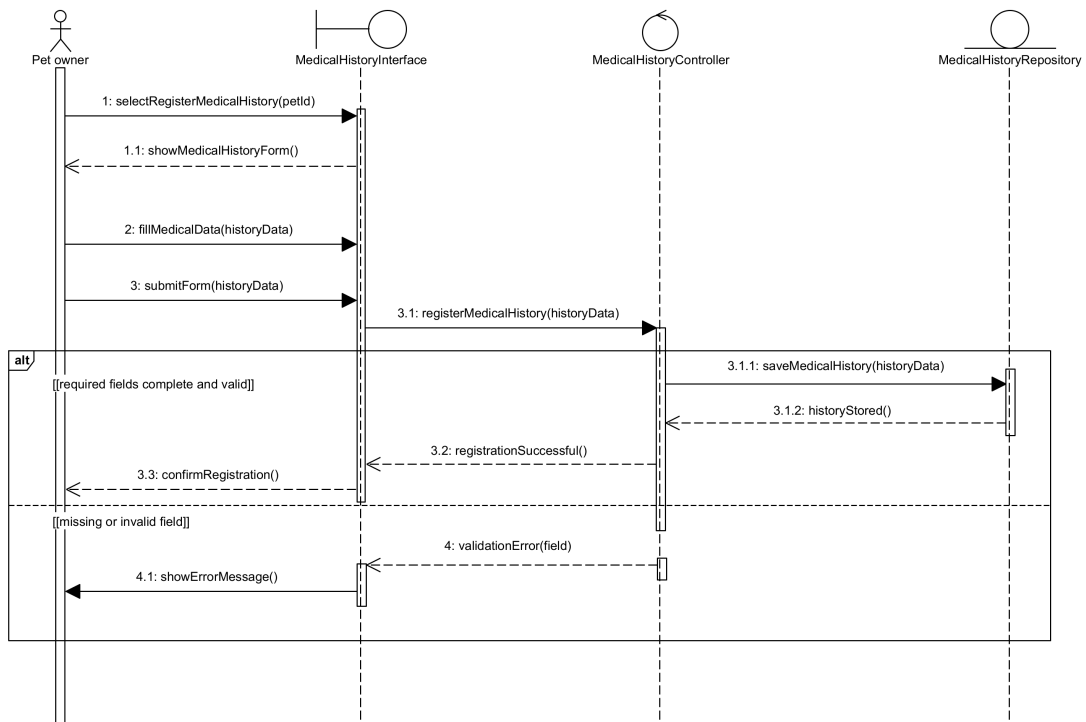


Figura 12: Diagrama de secuencia – Registrar historial médico y antecedentes genéticos (CU-02).

La estructura casi idéntica al CU-01 (mismo patrón de *alt* con validación en el controlador), visible en la Figura 12, refleja una decisión consciente de reutilizar el mismo patrón arquitectónico de registro para todo dato del sistema, de forma que el equipo pueda mantener consistencia entre módulos distintos (mascota vs. historial médico) sin duplicar convenciones. Esto evidencia que el historial médico se trata como una entidad de registro independiente y no como un campo más del perfil de la mascota.

1.8.3 Cargar documento veterinario y carnet de vacunación (CU-03)

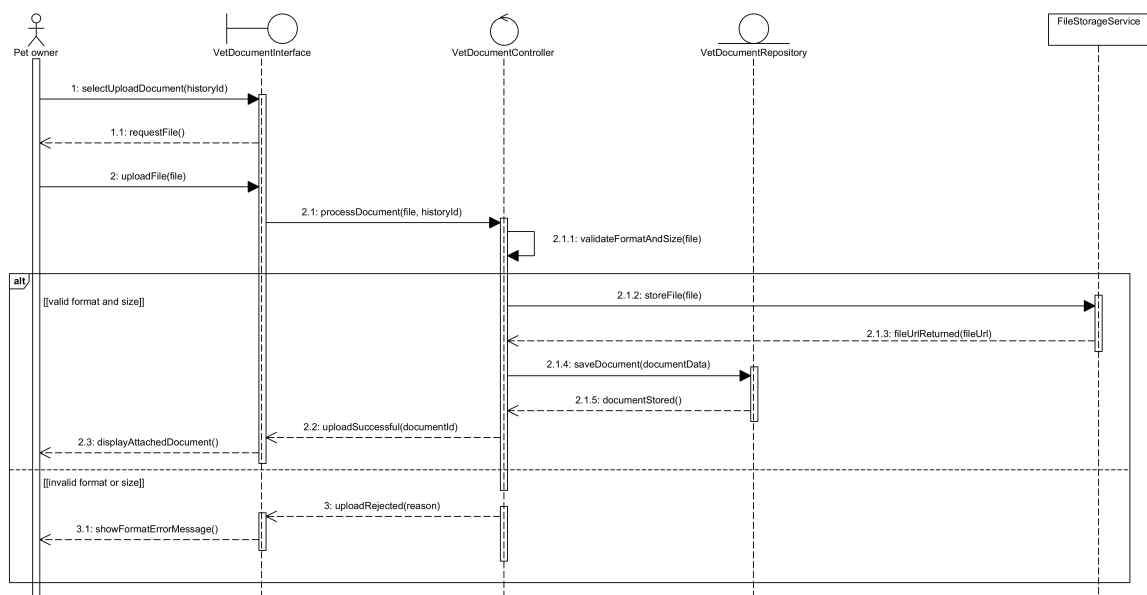


Figura 13: Diagrama de secuencia – Cargar documento veterinario y carnet de vacunación (CU-03).

Que *validateFormatAndSize(file)* sea una llamada reflexiva del propio *VetDocumentController* (mensaje a sí mismo) antes de invocar al *FileStorageService*, como se ve en la Figura 13, revela que el sistema nunca envía

un archivo a almacenamiento externo sin antes garantizar localmente que cumple formato y tamaño, evitando cargas costosas o inseguras a un servicio de terceros. También, que *storeFile* preceda a *saveDocument* (primero el archivo físico, luego el registro en base de datos) muestra que el diseño prioriza tener la evidencia física respaldada antes de dejar constancia lógica de que el documento existe.

1.8.4 Publicar mascota en adopción (CU-04)

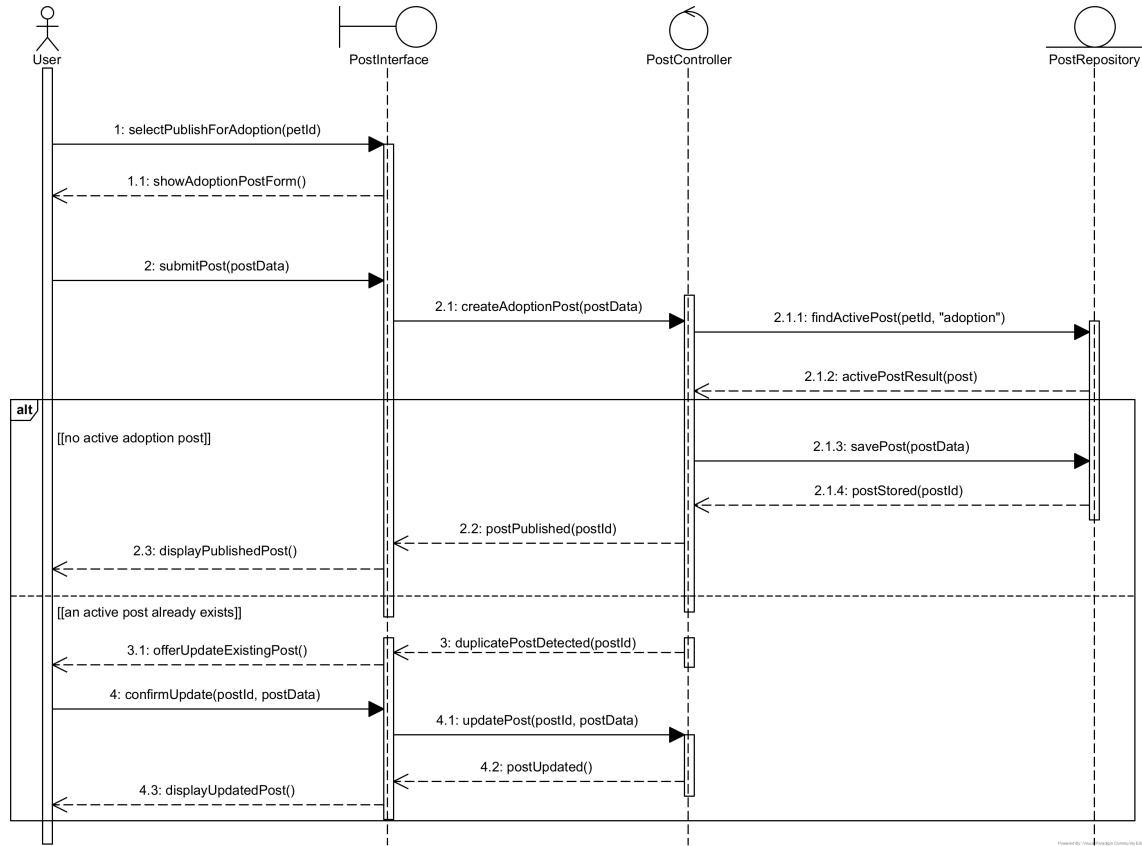


Figura 14: Diagrama de secuencia – Publicar mascota en adopción (CU-04).

La verificación *findActivePost(petId, "adoption")* antes de decidir entre crear o actualizar, presente en la Figura 14, revela que el sistema previene activamente publicaciones duplicadas de una misma mascota, tratando la unicidad de la publicación activa como una regla de negocio explícita y no como una validación de interfaz. Que el flujo de “an active post already exists” ofrezca actualizar en lugar de simplemente rechazar el nuevo intento (*offerUpdateExistingPost*) muestra que el diseño prioriza la conveniencia del usuario sobre el rechazo estricto.

1.8.5 Publicar mascota para cruce y gestionar solicitud (CU-05)

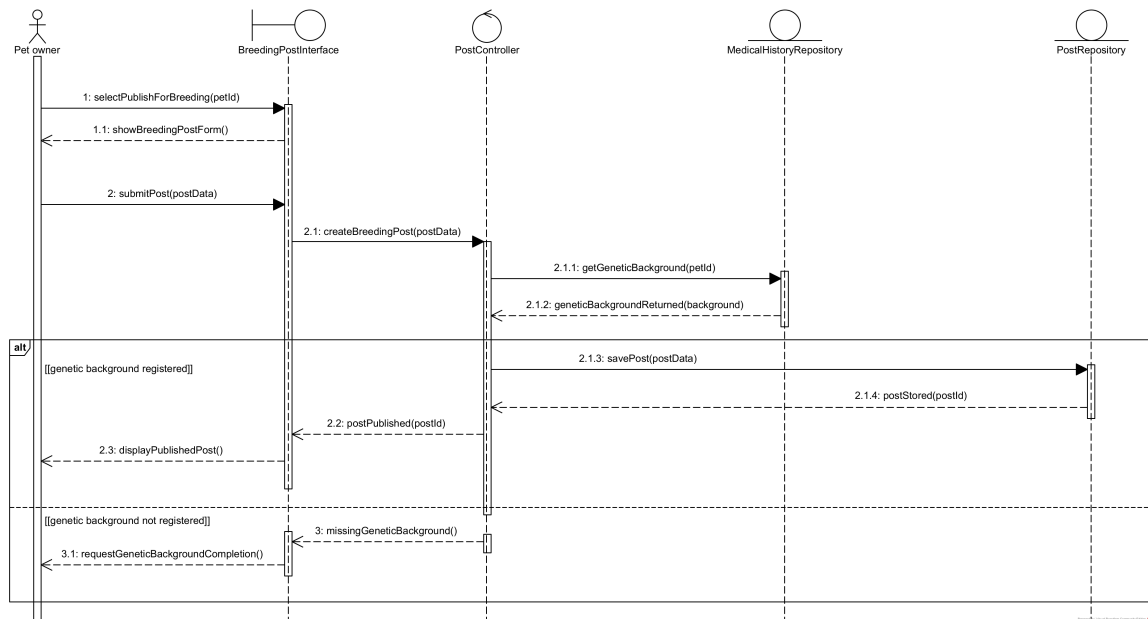


Figura 15: Diagrama de secuencia – Publicar mascota para cruce y gestionar solicitud (CU-05).

Que *getGeneticBackground(petId)* se consulte obligatoriamente antes de poder publicar (y bloquee la publicación en el camino “genetic background not registered”), como se observa en la Figura 15, revela que la trazabilidad genética no es un campo opcional del formulario, sino una precondition dura para la cruce, materializando directamente el requisito de “responsabilidad en la cruce” declarado en el alcance de MundiPets. Esta dependencia obligatoria contrasta con el CU-04 (adopción), donde no existe tal barrera, mostrando que ambos procesos de publicación tienen niveles de exigencia distintos según el riesgo asociado.

1.8.6 Configurar privacidad médica (CU-06)

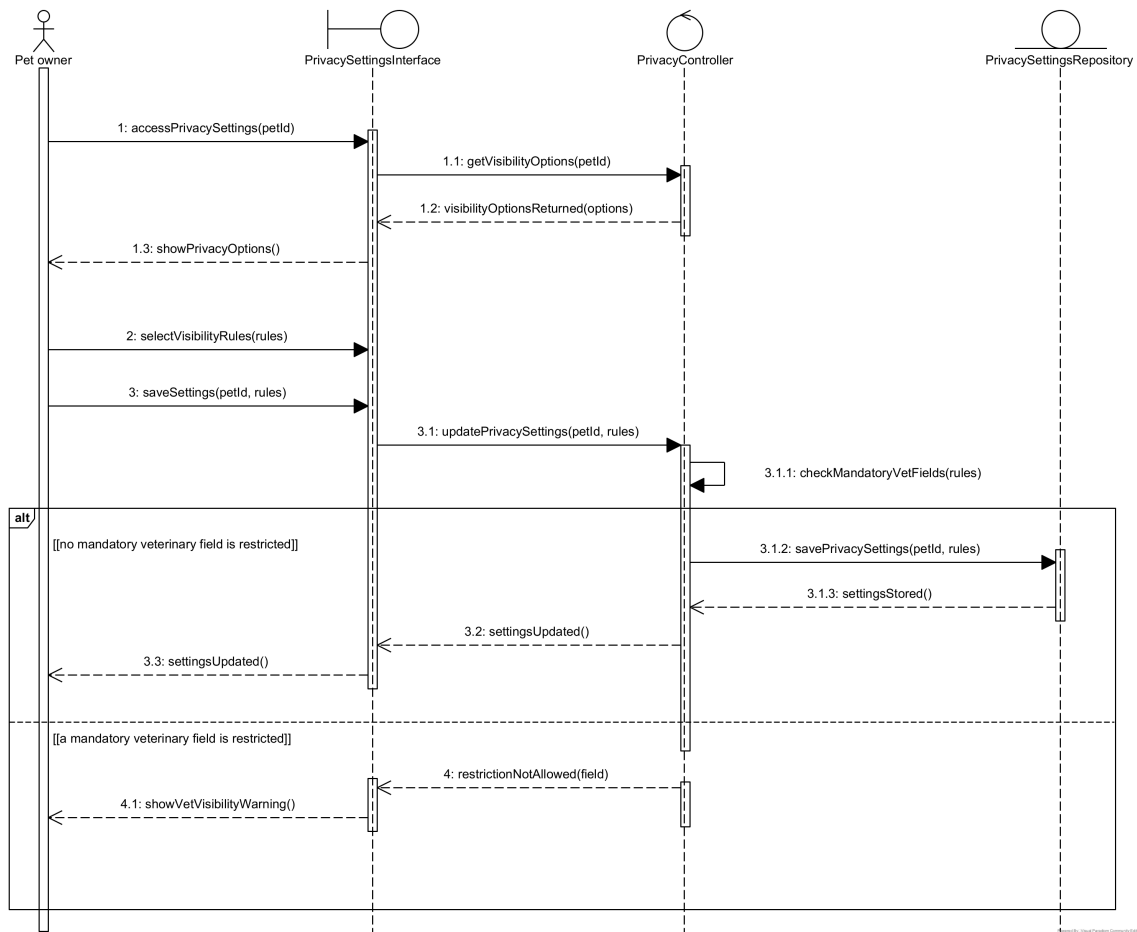


Figura 16: Diagrama de secuencia – Configurar privacidad médica (CU-06).

Que *checkMandatoryVetFields(rules)* sea una validación reflexiva del controlador que puede rechazar la solicitud del dueño (“a mandatory veterinary field is restricted”), tal como se ve en la Figura 16, revela que la privacidad configurable por el usuario tiene un límite impuesto por el sistema: el dueño puede ocultar información no crítica, pero no puede restringir campos que la clínica veterinaria necesita ver para validar. Esto demuestra que la privacidad del dato médico no es absoluta, sino que está subordinada a la seguridad clínica del proceso.

1.8.7 Gestionar proceso de adopción (CU-07)

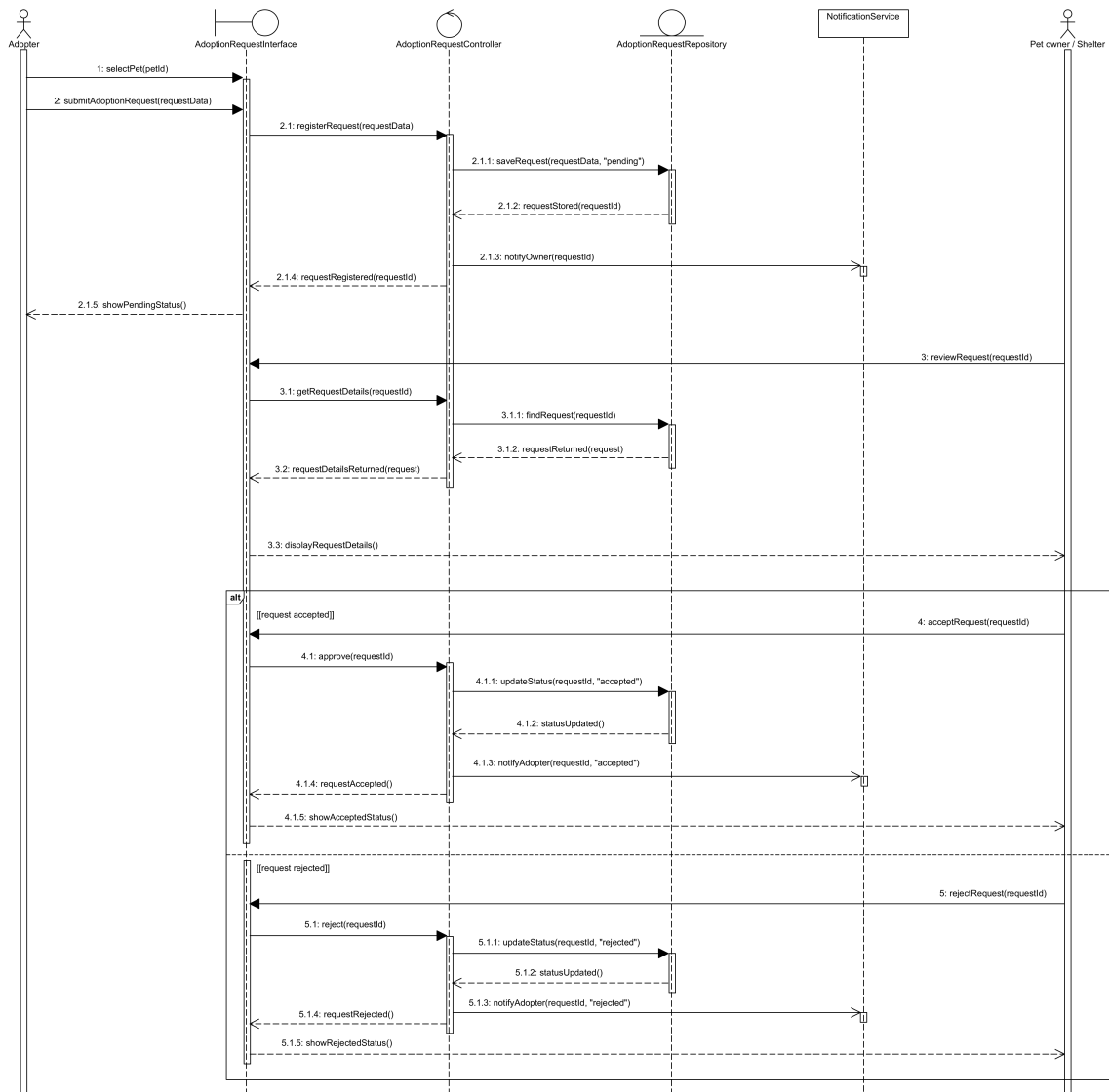


Figura 17: Diagrama de secuencia – Gestionar proceso de adopción (CU-07).

Que *notifyOwner/notifyAdopter* se disparen en cada transición de estado (registro, aceptación, rechazo) a través de un *NotificationService* externo y compartido, en lugar de que cada actor consulte manualmente el estado, según se ve en la Figura 17, revela que el proceso de adopción se diseñó como orientado a eventos: ningún cambio de estado ocurre en silencio. Que el *alt* solo contemple “accepted” o “rejected” como desenlaces (sin una rama de “cancelado” en este mismo diagrama) muestra que la cancelación se modela por separado (coherente con la máquina de estados de Solicitudes, Figura 9), evitando que este diagrama de secuencia intente cubrir cada excepción del ciclo de vida completo.

1.8.8 Gestionar comunicación entre usuarios (CU-08)

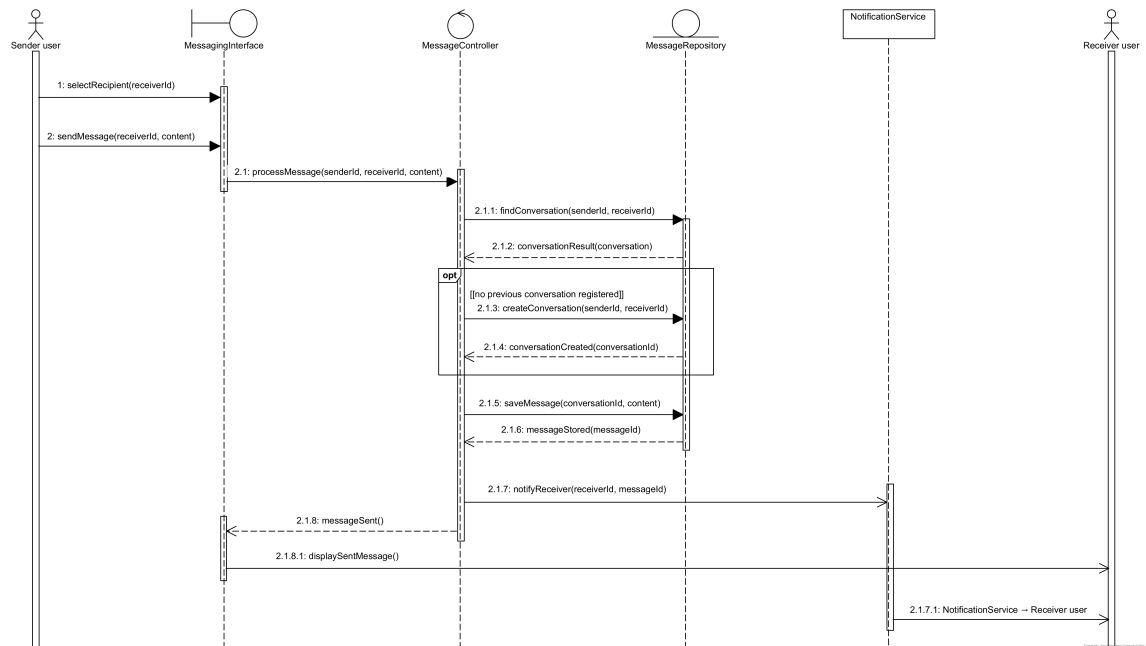


Figura 18: Diagrama de secuencia – Gestionar comunicación entre usuarios (CU-08).

Que *createConversation* esté envuelto en un bloque *opt* condicionado a “no previous conversation registered” (y no se ejecute siempre), como se observa en la Figura 18, revela que el sistema modela la conversación como una entidad persistente y reutilizable entre dos usuarios, evitando fragmentar el historial de mensajes en hilos nuevos cada vez que alguien escribe. Que *notifyReceiver* ocurra después de *messageStored* (nunca antes) muestra que el sistema garantiza la persistencia del mensaje como condición previa a cualquier notificación, evitando alertar de algo que aún podría fallar al guardarse.

1.8.9 Validar información médica (CU-09)

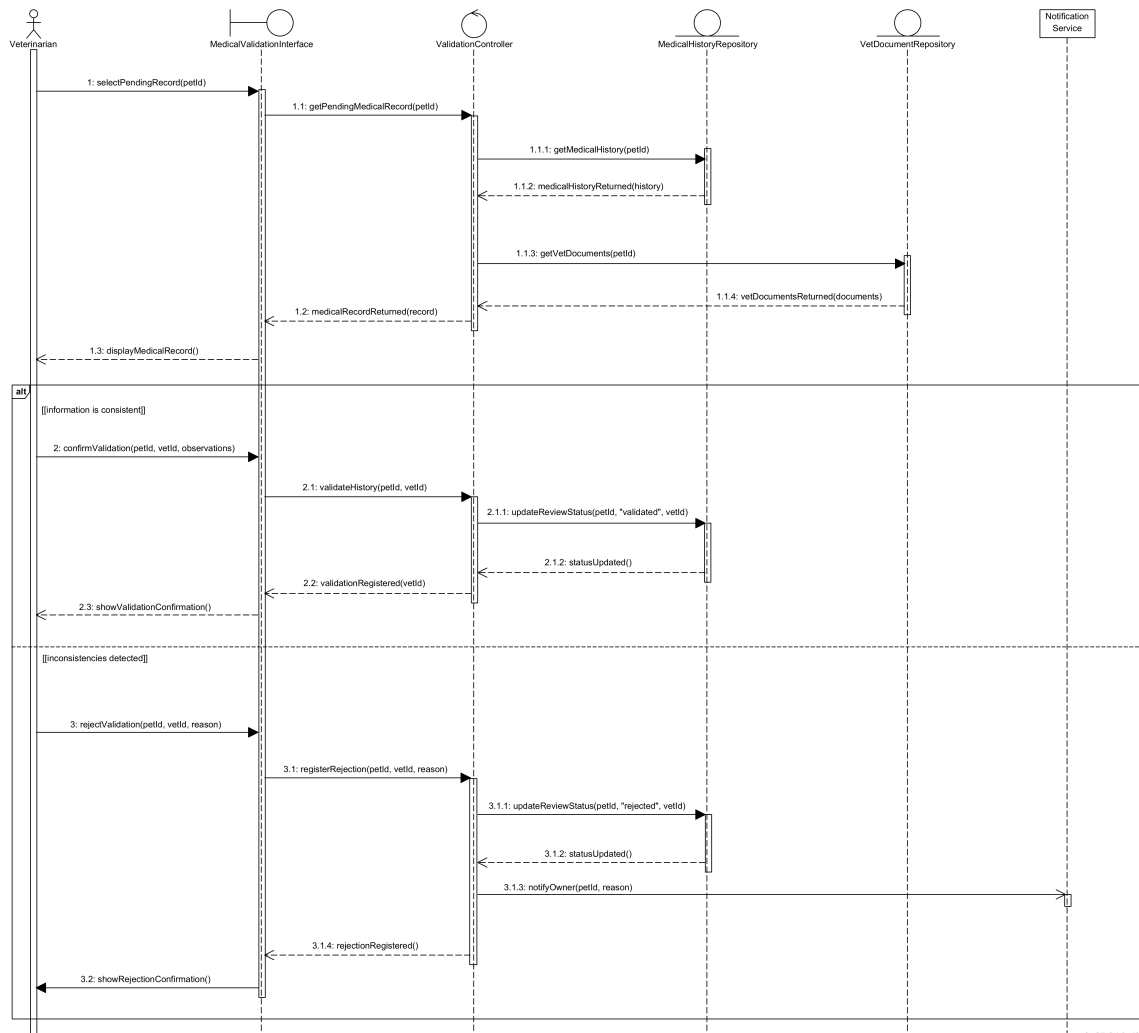


Figura 19: Diagrama de secuencia – Validar información médica (CU-09).

Que el veterinario reciba tanto *getMedicalHistory* como *getVetDocuments* en paralelo desde dos repositorios distintos antes de emitir cualquier veredicto, tal como se ve en la Figura 19, revela que la validación clínica exige cruzar la información declarada por el dueño con el documento formal adjunto, y no aceptar ninguna de las dos fuentes de forma aislada. Que el *alt* solo ofrezca “validated” o “rejected” (sin estado intermedio) y que el rechazo dispare *notifyOwner(petId, reason)* con motivo explícito, refleja que el sistema exige trazabilidad del porqué de un rechazo, no solo el hecho de que ocurrió, coherente con la máquina de estados de Verificación de Documentos (Figura 10).

1.8.10 Consultar perfil médico (CU-10)

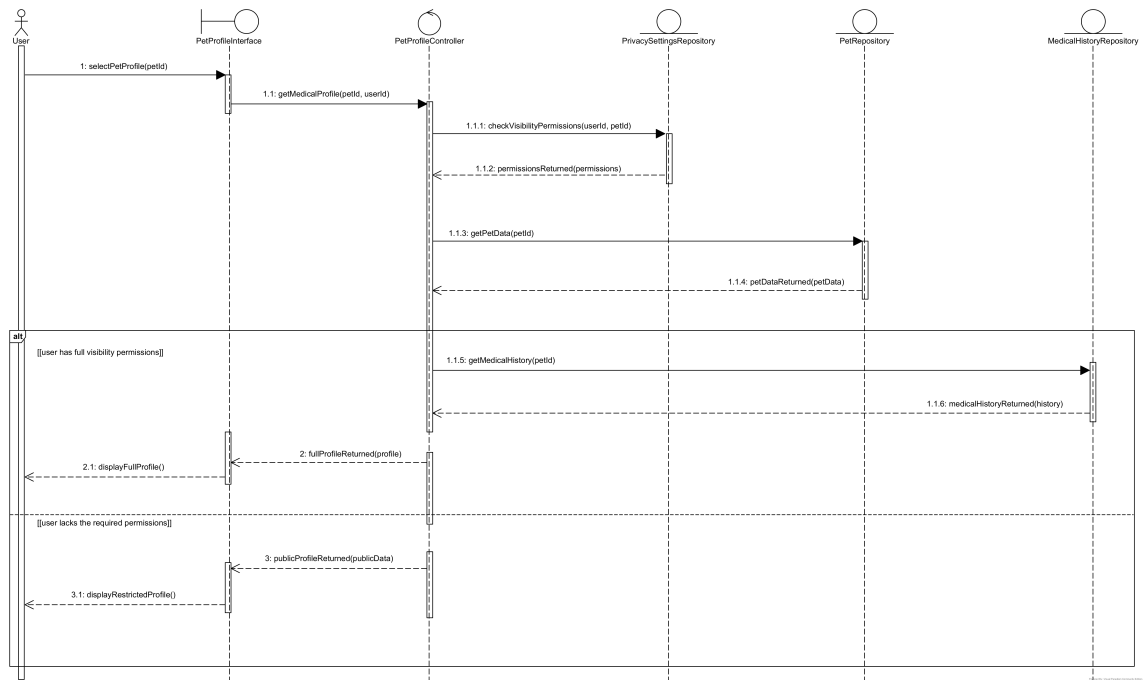


Figura 20: Diagrama de secuencia – Consultar perfil médico (CU-10).

Que *checkVisibilityPermissions(userId, petId)* se ejecute antes de decidir si se llama a *getMedicalHistory* (el *alt* separa “full visibility permissions” de “lacks the required permissions”), como se observa en la Figura 20, revela que el perfil médico no es un dato binario visible/oculto, sino que el propio controlador construye una respuesta distinta según el solicitante: la información sensible ni siquiera se consulta si el usuario no tiene permiso, en vez de consultarla y luego filtrarla. Esto materializa directamente el CU-06 (Figura 16): ambos diagramas comparten la misma fuente de reglas (*PrivacySettingsRepository*).

1.8.11 Buscar mascota disponible (CU-11)

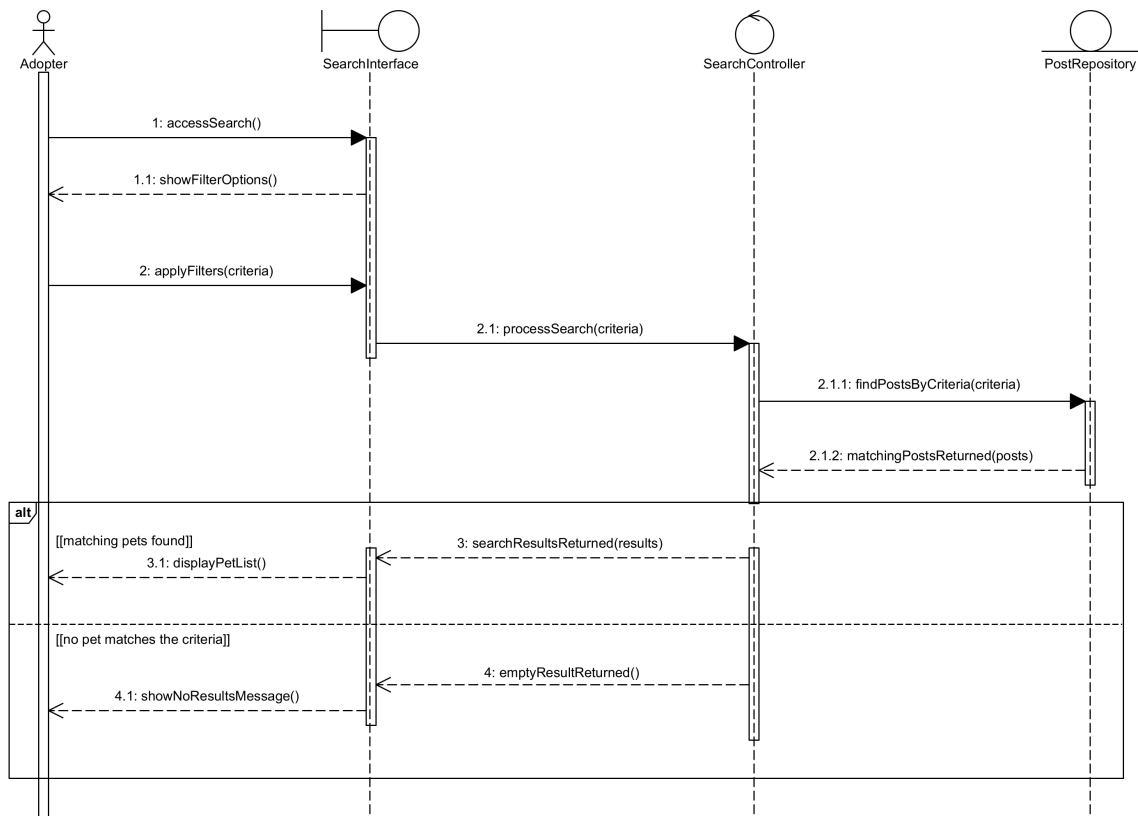


Figura 21: Diagrama de secuencia – Buscar mascota disponible (CU-11).

Que el *alt* distinga entre “matching pets found” y “no pet matches the criteria” como dos resultados igualmente válidos de una búsqueda (y no un error), tal como se ve en la Figura 21, revela que el sistema trata la ausencia de resultados como un estado de negocio esperado y comunicable (*showNoResultsMessage*), no como una excepción a capturar. La simplicidad del flujo (sin bloque de validación de errores de entrada) muestra que este caso de uso confía en que los filtros de búsqueda siempre producen criterios sintácticamente válidos, delegando esa validación a la capa de interfaz.

1.8.12 Consultar recomendaciones de cuidado (CU-12)

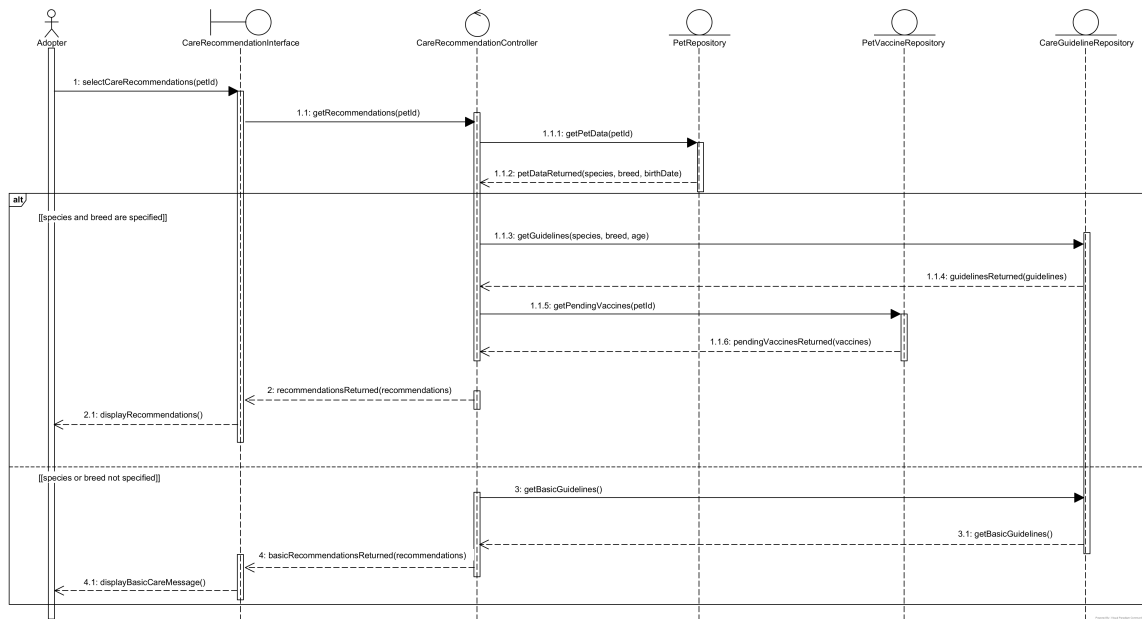


Figura 22: Diagrama de secuencia – Consultar recomendaciones de cuidado (CU-12).

Que el *alt* dependa de si “species and breed are specified” y no de si el usuario las proporcionó explícitamente en este momento (los datos ya vienen de *getPetData*), como se observa en la Figura 22, revela que las recomendaciones de cuidado dependen de la calidad de los datos previamente registrados del perfil, no de una nueva entrada del adoptante: si el perfil está incompleto, el sistema degrada a *getBasicGuidelines()* en lugar de bloquear la función. Esta decisión de tener un camino de *fallback* con recomendaciones genéricas, en vez de negar el servicio, refleja que la utilidad de la funcionalidad se prioriza sobre la exhaustividad de los datos.

1.8.13 Evaluar compatibilidad de cruce (CU-13)

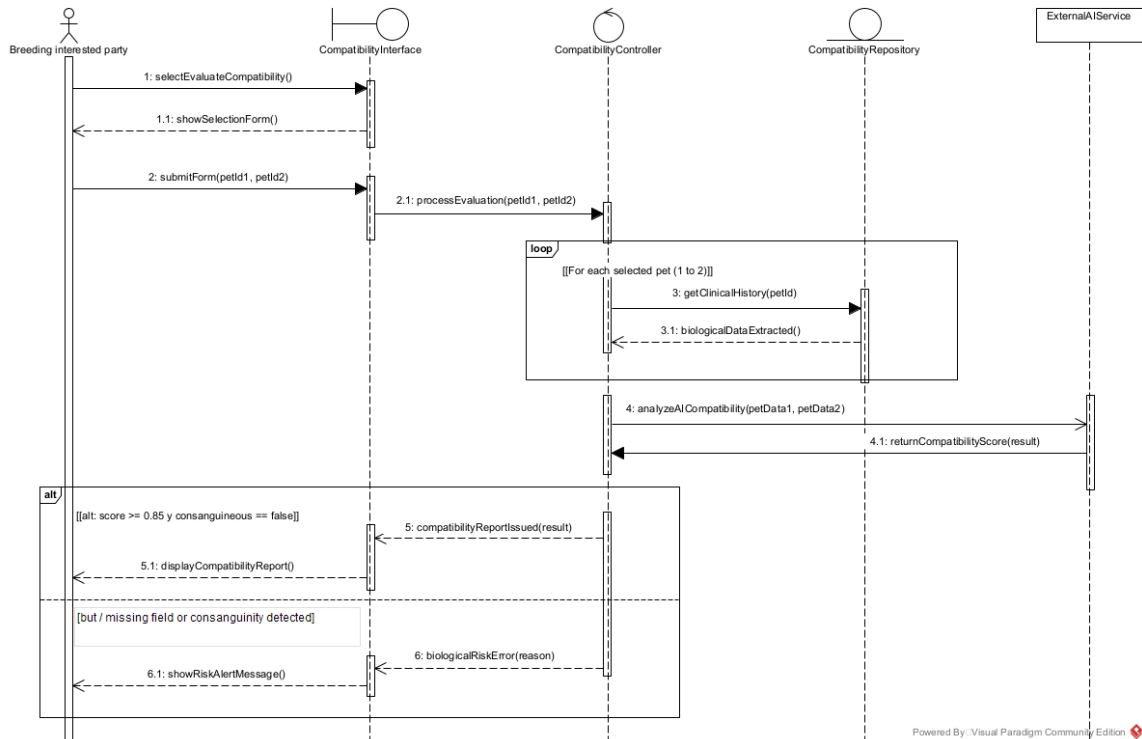


Figura 23: Diagrama de secuencia – Evaluar compatibilidad de cruce (CU-13).

Que el *loop* recorra los historiales clínicos de ambas mascotas (1 a 2) antes de invocar al *ExternalAIService*, y que la condición del *alt* combine dos criterios distintos (*score* ≥ 0.85 y *consanguineous* $== false$) en vez de basarse solo en el puntaje de la IA, tal como se ve en la Figura 23, revela que la recomendación de un componente probabilístico nunca es la única barrera de decisión: existe una regla determinista de consanguinidad que puede anular un buen puntaje de compatibilidad. Esto es la aplicación directa del límite declarado en la Sección ?? (“no reemplaza la evaluación veterinaria profesional”): la IA aporta una señal, pero una regla de negocio dura puede vetarla.

1.9 Diagramas de Actividad

1.9.1 Registrar mascota (CU-01)

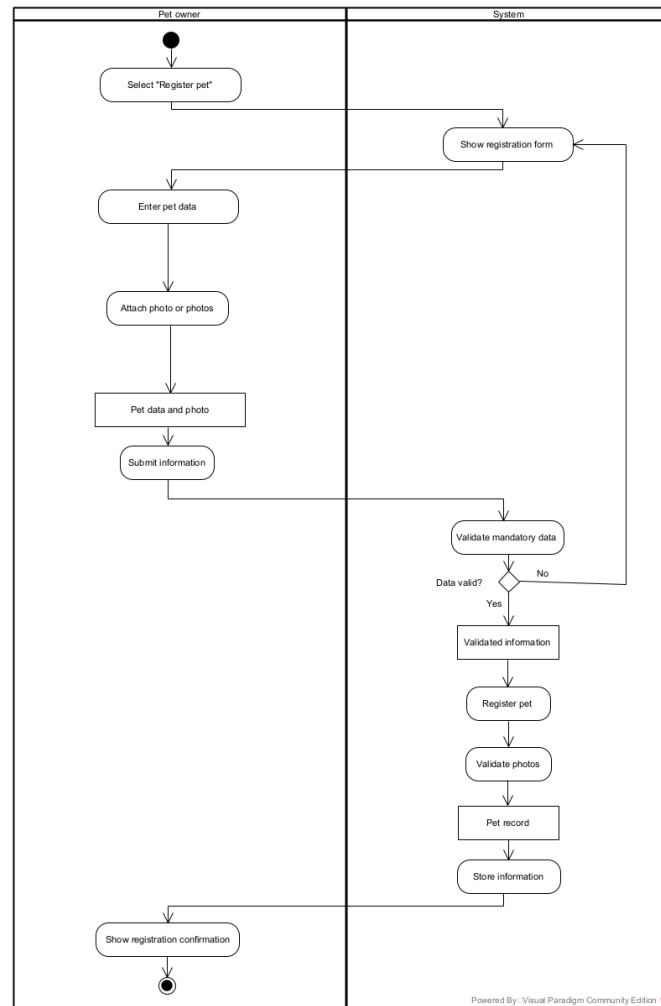


Figura 24: Diagrama de actividades – Registrar mascota (CU-01).

Que la validación “Data valid?” con salida “No” regrese al inicio del carril *System* (“Show registration form”) y no simplemente muestre un mensaje de error puntual, como se ve en la Figura 24, revela que el sistema fuerza al dueño a repetir el ciclo completo de llenado en lugar de corregir solo el campo fallido, priorizando la simplicidad de implementación sobre la fricción del usuario en esta primera versión del flujo.

1.9.2 Registrar historial médico y antecedentes genéticos (CU-02)

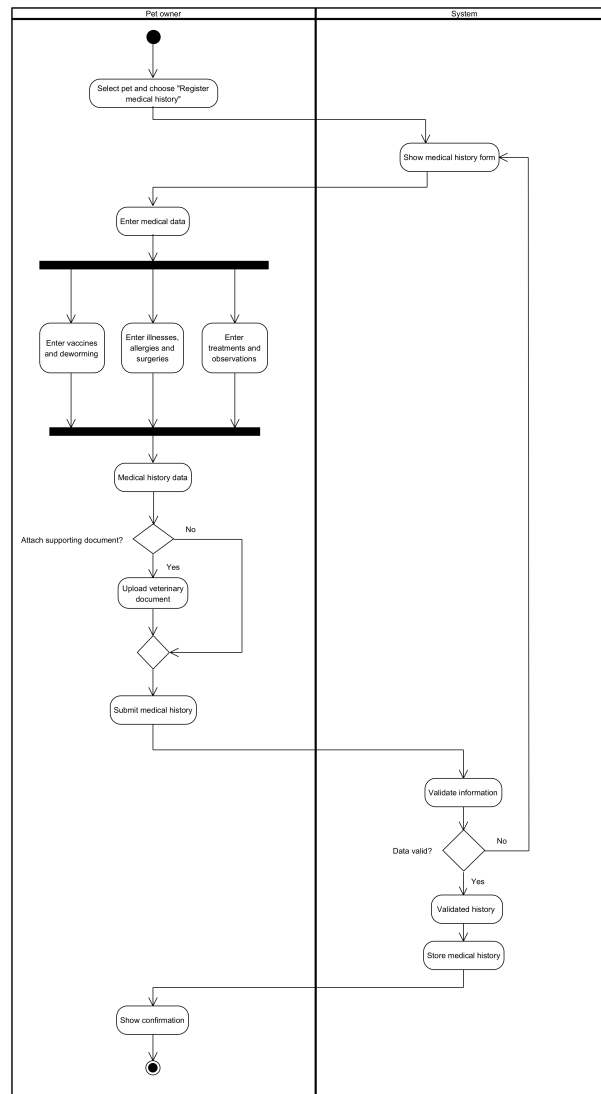


Figura 25: Diagrama de actividades – Registrar historial médico y antecedentes genéticos (CU-02).

Que “Enter vaccines and deworming”, “Enter illnesses...” y “Enter treatments...” se ejecuten en una bifurcación paralela (*fork/join*) y no en secuencia, según se observa en la Figura 25, revela que estas tres categorías de datos médicos son independientes entre sí y no existe un orden obligatorio de captura, permitiendo que el dueño complete primero la que tenga más a mano. Que “Attach supporting document?” sea una decisión opcional embebida dentro de este mismo caso de uso (y no delegada por completo al CU-03) muestra que subir el documento formal es una posibilidad conveniente durante el registro, aunque el CU-03 exista como flujo independiente para hacerlo después.

1.9.3 Cargar documento veterinario y carnet de vacunación (CU-03)

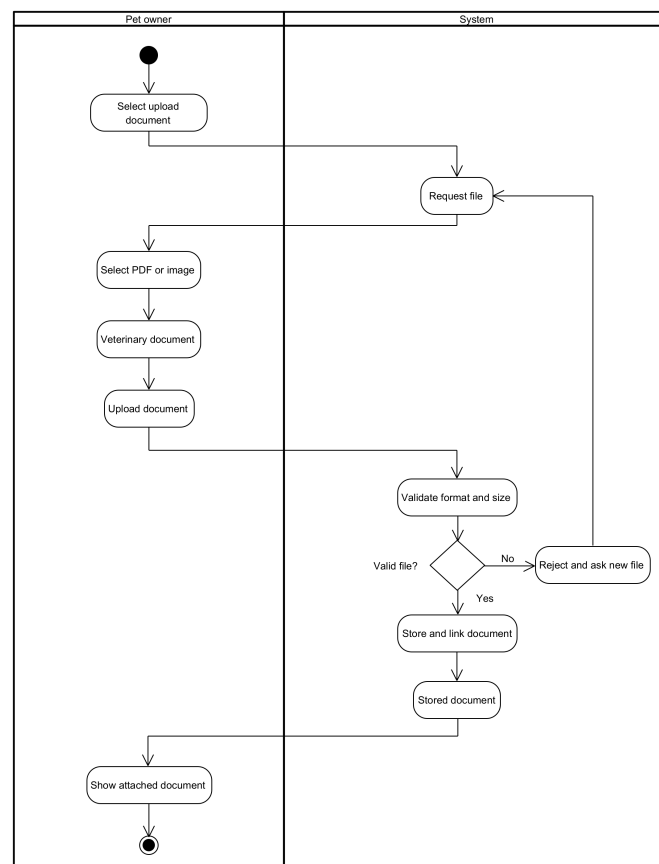


Figura 26: Diagrama de actividades – Cargar documento veterinario y carnet de vacunación (CU-03).

Que el rechazo del archivo (“Reject and ask new file”) regrese al nodo “Request file” (y no obligue a reiniciar todo el caso de uso desde “Select upload document”), tal como se ve en la Figura 26, revela que el sistema minimiza la fricción ante un archivo inválido, permitiendo reintentar solo el paso de selección del archivo sin perder el contexto de a qué historial médico pertenece.

1.9.4 Publicar mascota en adopción (CU-04)

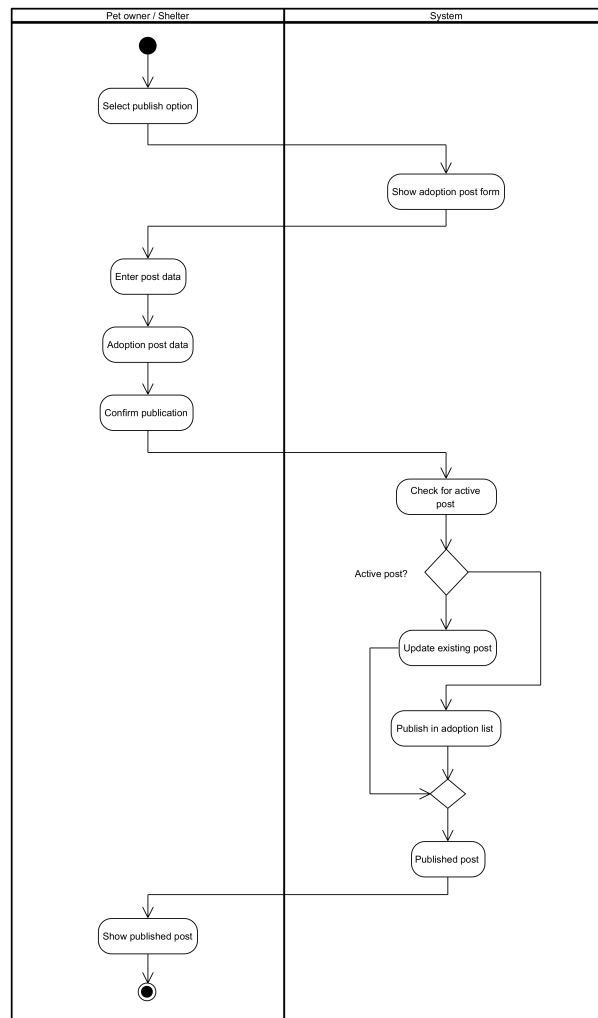


Figura 27: Diagrama de actividades – Publicar mascota en adopción (CU-04).

Que la decisión “Active post?” determine si se ejecuta “Update existing post” antes de fusionar el flujo hacia “Publish in adoption list” (ambas ramas convergen al mismo paso de publicación), como se observa en la Figura 27, revela que, exista o no una publicación previa, el resultado final es siempre una sola publicación activa vigente: el sistema absorbe la lógica de duplicados de forma transparente para el usuario, que solo ve el resultado publicado, sin pasos adicionales visibles según el caso.

1.9.5 Publicar mascota para cruce y gestionar solicitud (CU-05)

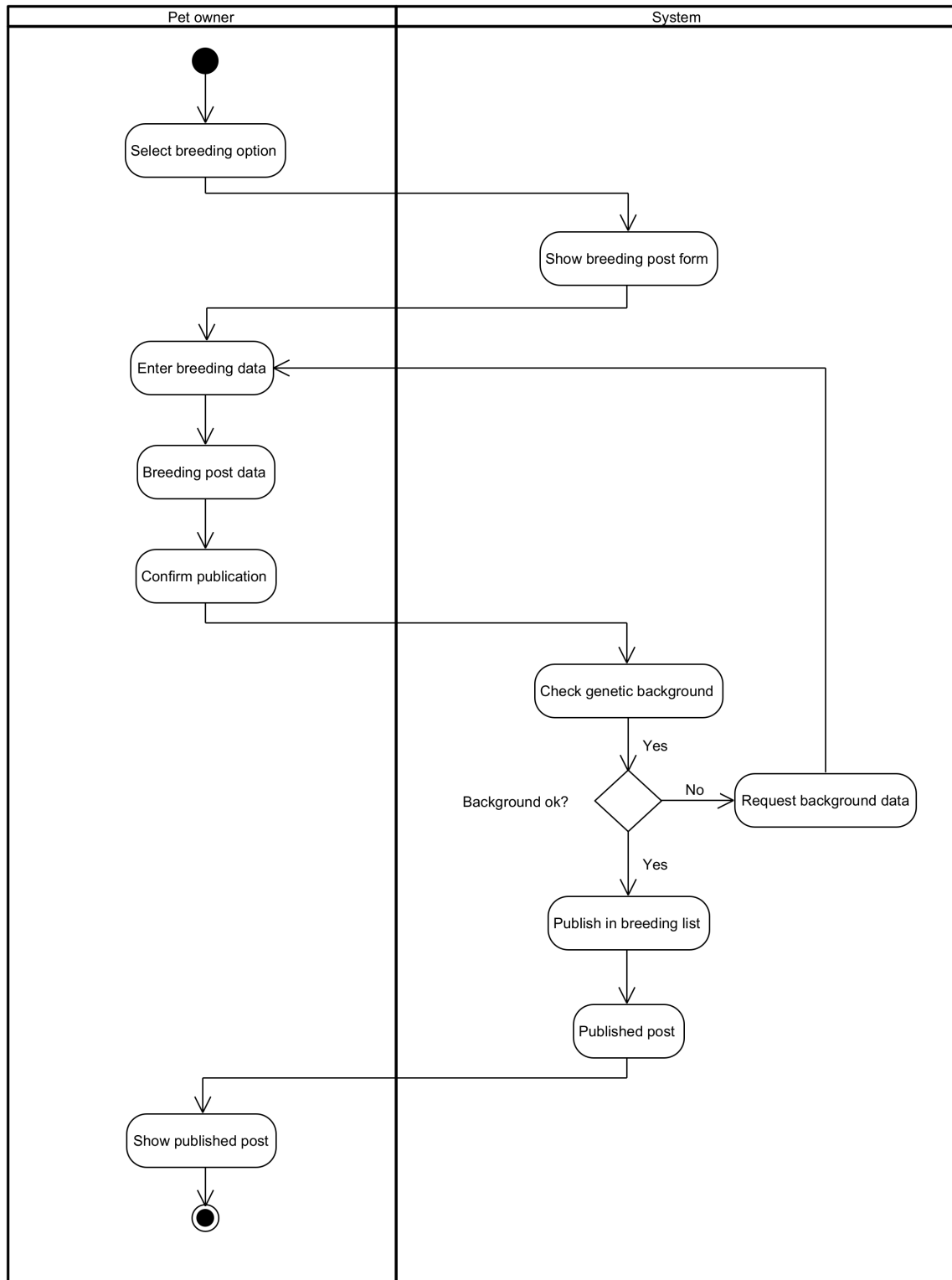


Figura 28: Diagrama de actividades – Publicar mascota para cruce y gestionar solicitud (CU-05).

Que “Background ok?” con salida “No” regrese específicamente a “Enter breeding data” (y no a “Show breeding post form” como en otros flujos), tal como se ve en la Figura 28, revela que el sistema no permite publicar sin importar cuántas veces se lo intente si falta el antecedente genético: el dueño queda atrapado en un ciclo de corrección hasta cumplir este requisito no negociable, reafirmando visualmente lo mismo que ya mostró el diagrama de secuencia CU-05 (Figura 15): la trazabilidad genética es condición dura, no sugerencia.

1.9.6 Configurar privacidad médica (CU-06)

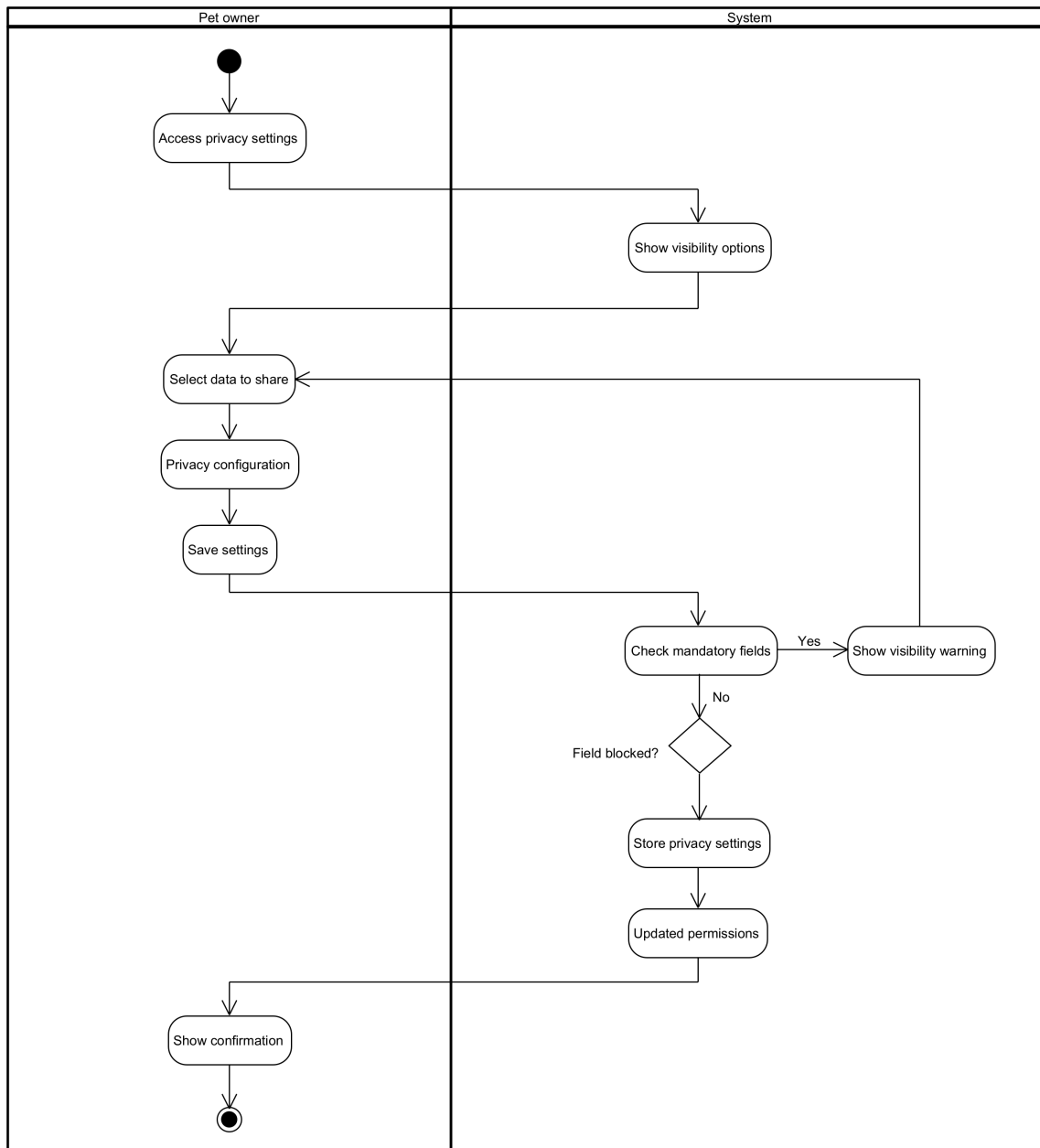


Figura 29: Diagrama de actividades – Configurar privacidad médica (CU-06).

Que “Check mandatory fields” con salida “Yes” (campo bloqueado indebidamente) regrese a “Select data to share” mediante “Show visibility warning” (en vez de simplemente no guardar el cambio en silencio), como se ve en la Figura 29, revela que el sistema educa activamente al dueño sobre por qué su configuración fue rechazada, en lugar de aplicar un filtro invisible que ignore la instrucción del usuario sin explicación.

1.9.7 Gestionar proceso de adopción (CU-07)

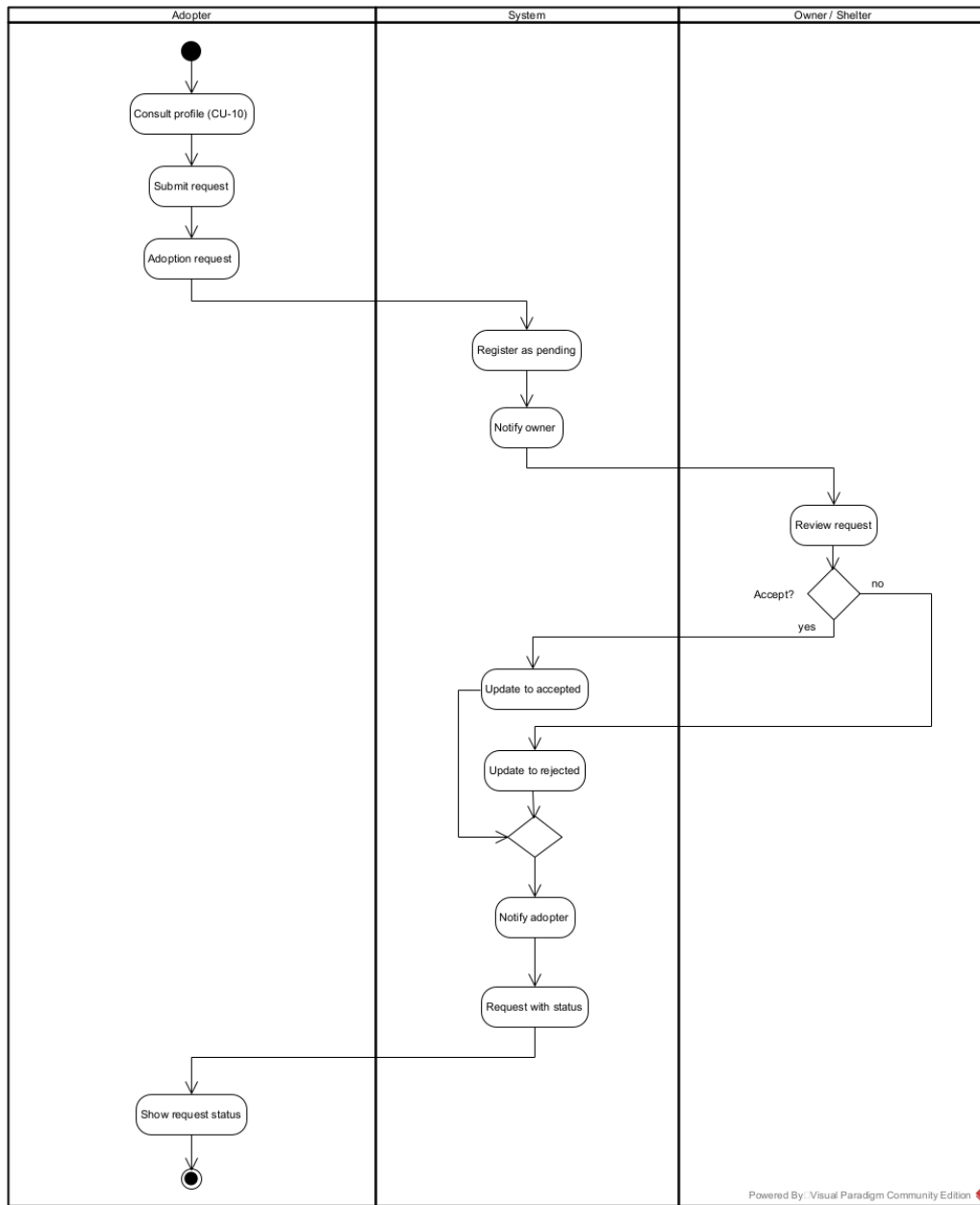


Figura 30: Diagrama de actividades – Gestionar proceso de adopción (CU-07).

Que exista un carril separado “Owner / Shelter” distinto del carril “System” para la actividad “Review request”, tal como se ve en la Figura 30, revela que la decisión de aceptar o rechazar una solicitud es exclusivamente humana y deliberada, sin ningún paso automatizado de aprobación: el sistema solo gestiona el registro y la notificación, nunca decide en nombre del dueño.

1.9.8 Gestionar comunicación entre usuarios (CU-08)

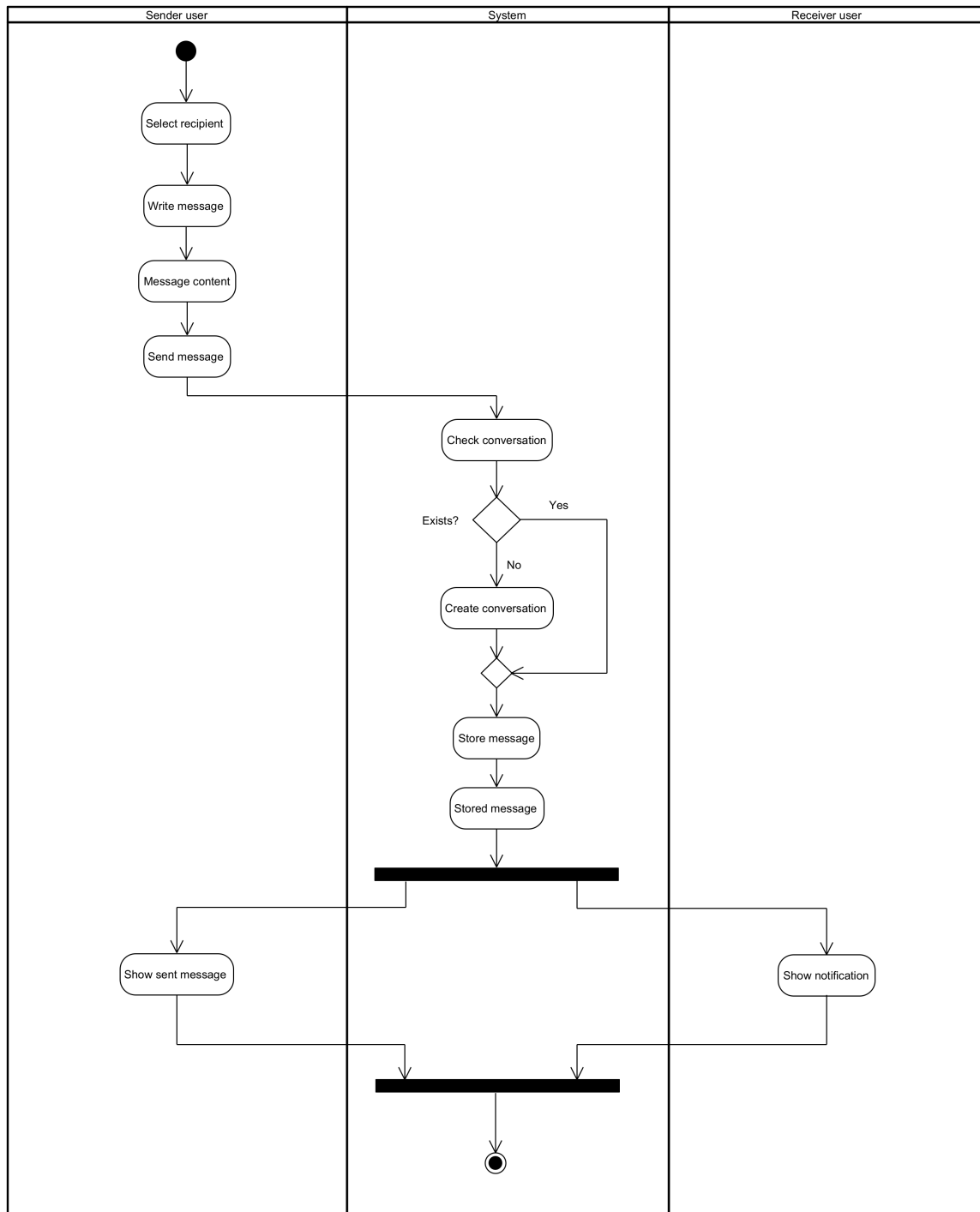


Figura 31: Diagrama de actividades – Gestionar comunicación entre usuarios (CU-08).

Que el diagrama cierre con una bifurcación (*fork*) que ejecuta en paralelo “Show sent message” (carril *Sender*) y “Show notification” (carril *Receiver*), como se observa en la Figura 31, revela que el sistema trata la confirmación del emisor y la alerta del receptor como dos efectos simultáneos e independientes de un mismo evento de guardado, y no como una cadena secuencial donde uno depende del otro.

1.9.9 Validar información médica (CU-09)

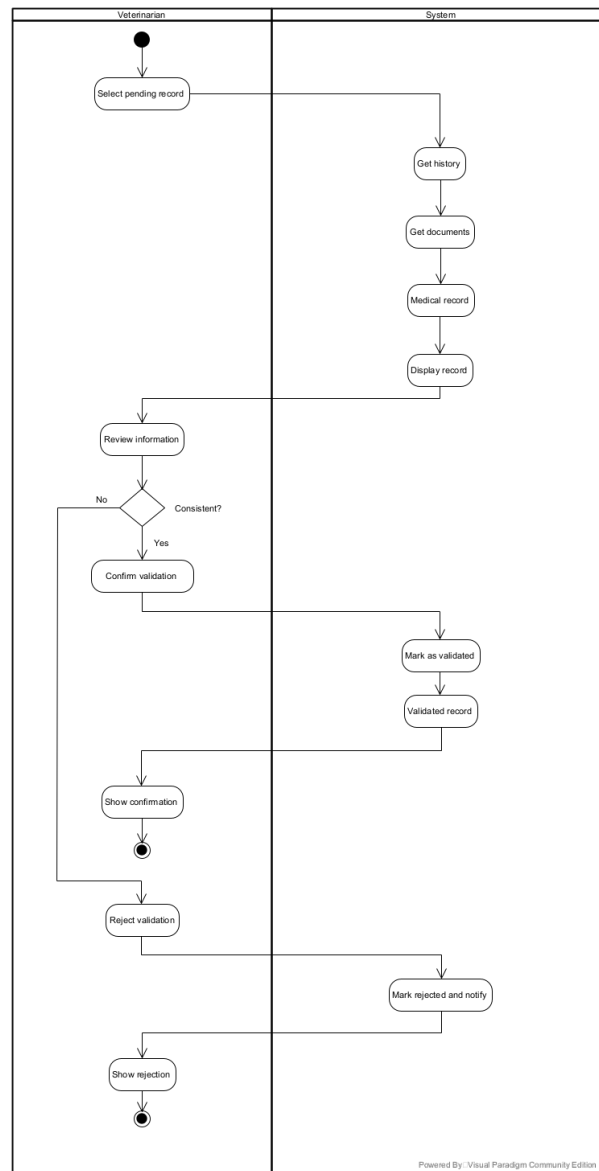


Figura 32: Diagrama de actividades – Validar información médica (CU-09).

Que el flujo de rechazo (“Reject validation”) no reingrese al ciclo de revisión sino que termine directamente en su propio nodo final (“Show rejection”), tal como se ve en la Figura 32, revela que, a diferencia del CU-05 o CU-06, esta actividad no fuerza una corrección inmediata: el veterinario simplemente cierra el caso como rechazado y es el dueño quien deberá iniciar un nuevo ciclo de validación por separado, coherente con la máquina de estados de Verificación de Documentos (Figura 10).

1.9.10 Consultar perfil médico (CU-10)

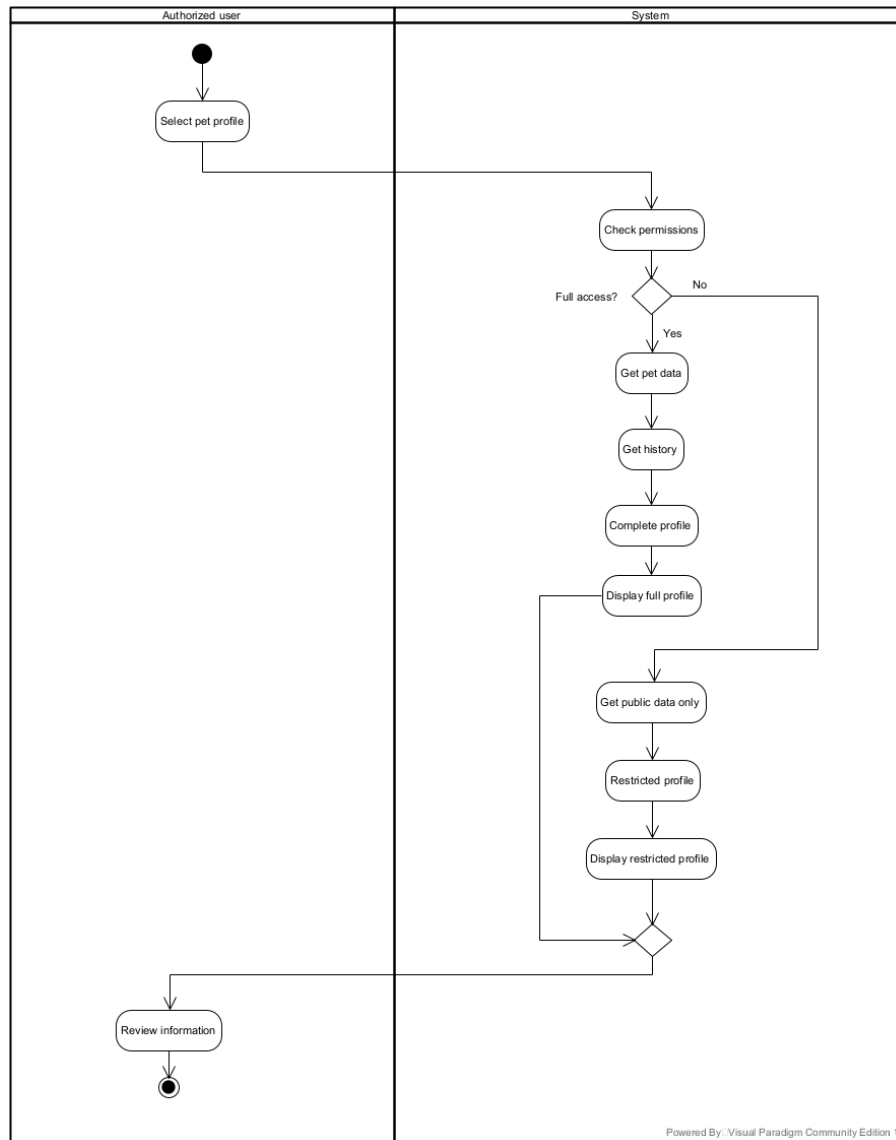
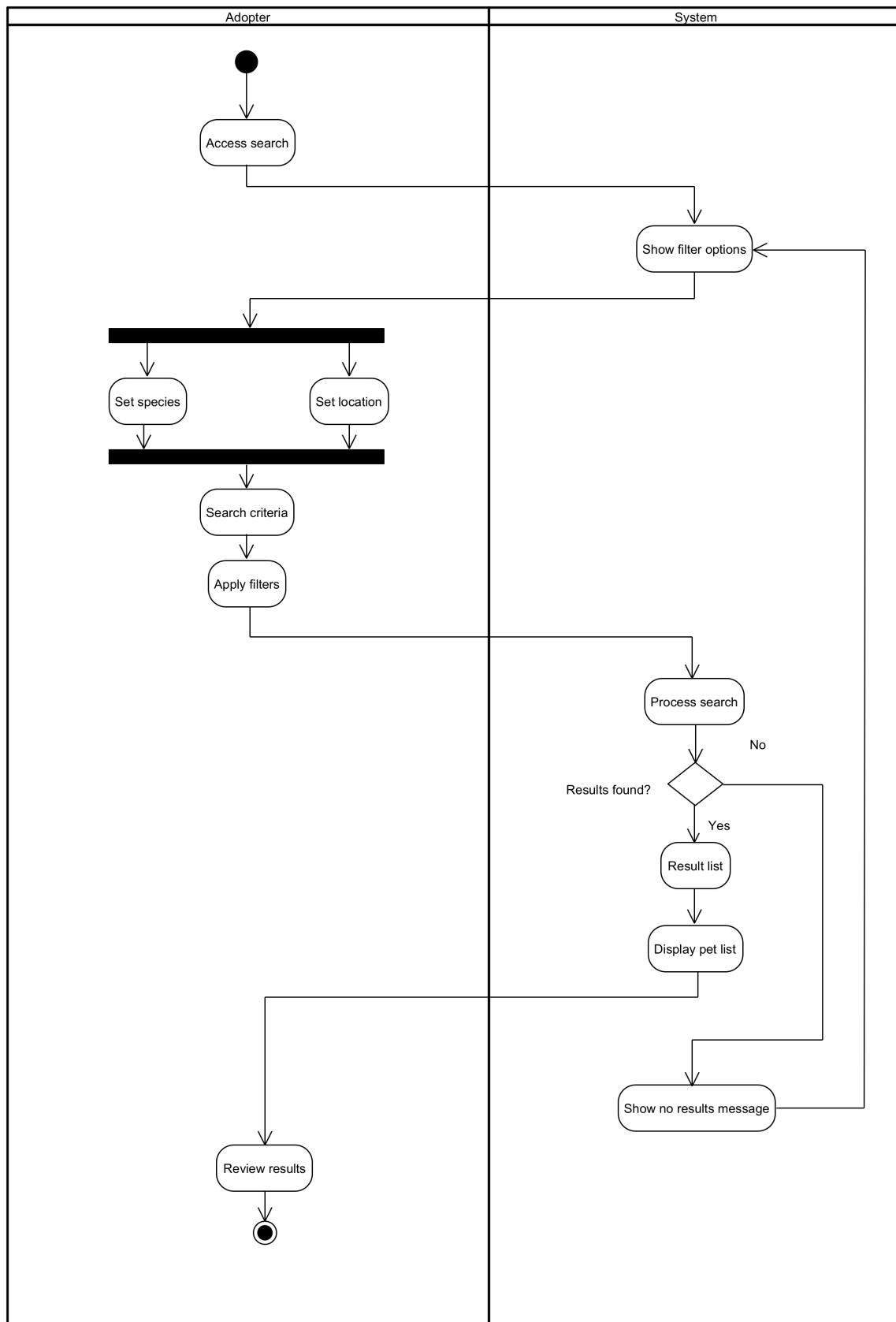


Figura 33: Diagrama de actividades – Consultar perfil médico (CU-10).

Que las dos ramas de la decisión “Full access?” (perfil completo vs. restringido) converjan en un mismo punto de unión antes de “Review information”, como se observa en la Figura 33, revela que, desde la perspectiva del usuario final, ambos caminos son indistinguibles en la interfaz: siempre recibe “un perfil”, solo que su contenido varía según el permiso, reforzando que la restricción ocurre completamente del lado del sistema y nunca se expone como un estado de error al usuario.

1.9.11 Buscar mascota disponible (CU-11)**Figura 34:** Diagrama de actividades – Buscar mascota disponible (CU-11).

Que “Set species” y “Set location” se definan como actividades paralelas (*fork/join*) dentro de “Apply filters”, tal como se ve en la Figura 34, revela que los criterios de búsqueda son independientes entre sí y combinables libremente, sin que uno dependa de haber definido antes el otro, permitiendo búsquedas parciales (solo especie, o solo ubicación).

1.9.12 Consultar recomendaciones de cuidado (CU-12)

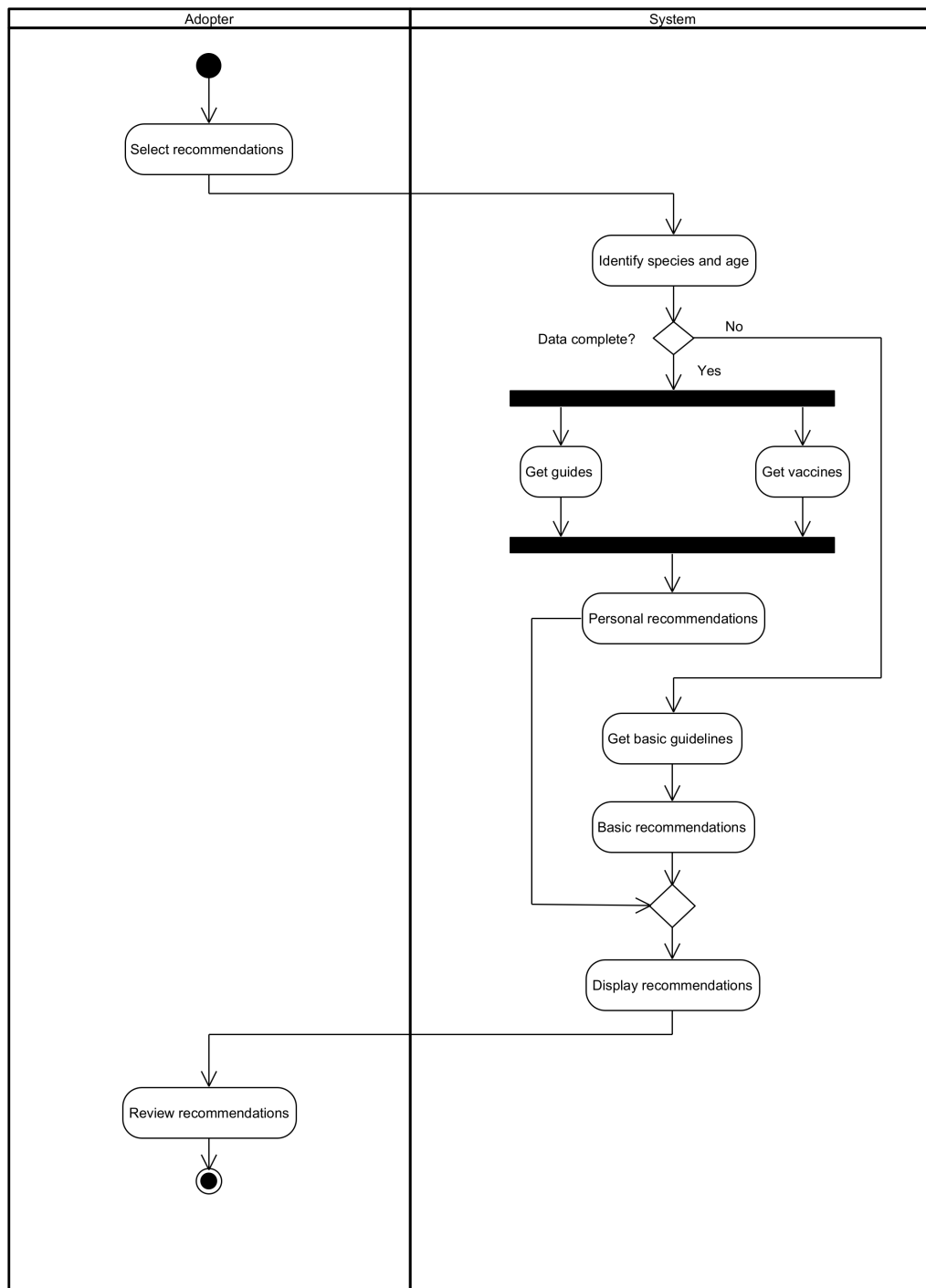


Figura 35: Diagrama de actividades – Consultar recomendaciones de cuidado (CU-12).

Que “Data complete?” con salida “No” desvíe directamente hacia “Get basic guidelines” (evitando por completo la rama paralela de “Get guides”/“Get vaccines”), como se ve en la Figura 35, revela que el sistema no intenta forzar al adoptante a completar el perfil antes de dar alguna recomendación: prefiere entregar un resultado genérico útil de inmediato en lugar de bloquear la funcionalidad por datos incompletos, igual que se observó en el diagrama de secuencia correspondiente (Figura 22).

1.9.13 Evaluar compatibilidad de cruce (CU-13)

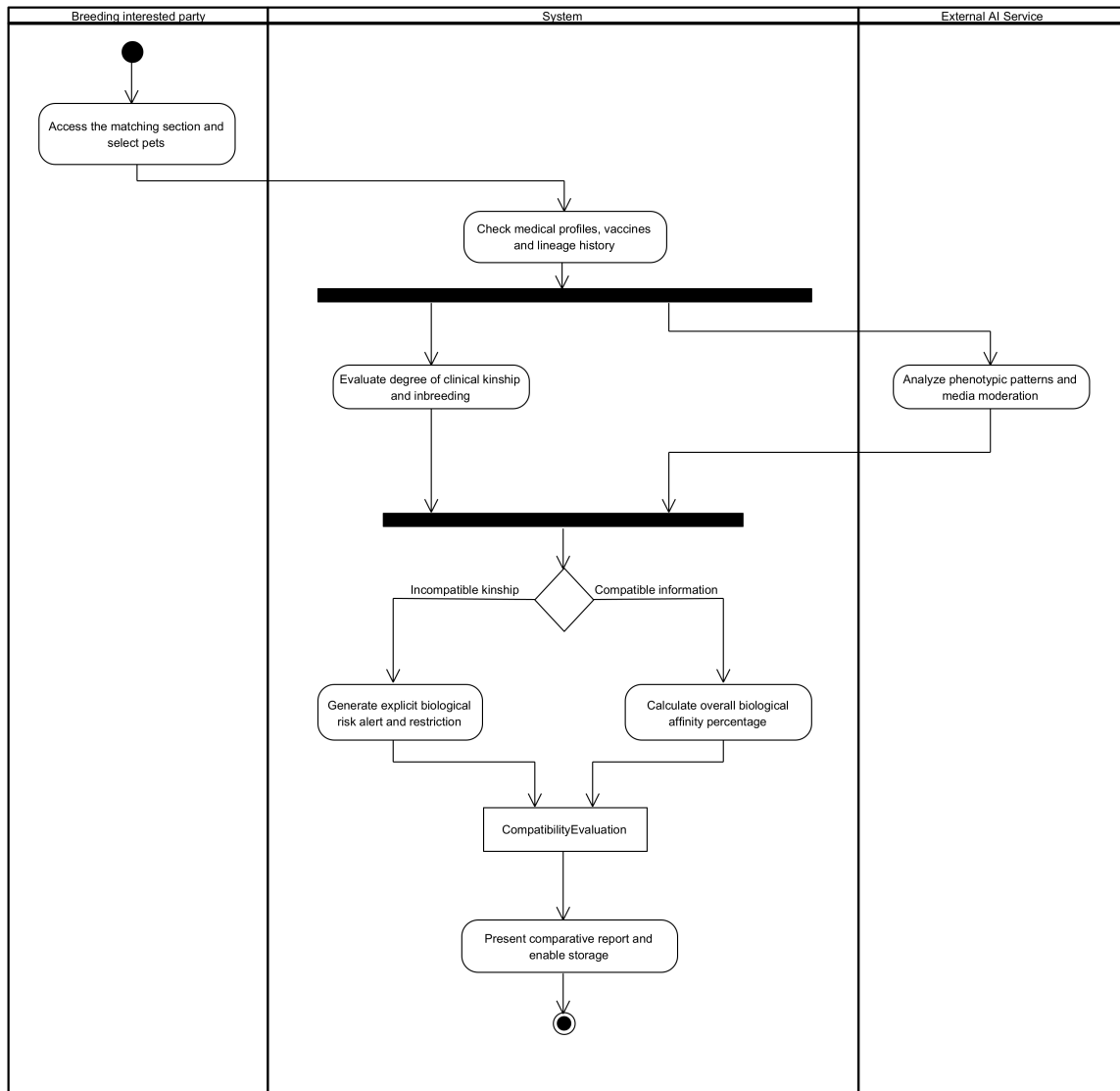


Figura 36: Diagrama de actividades – Evaluar compatibilidad de cruce (CU-13).

Que “Evaluate degree of clinical kinship and inbreeding” (carril *System*) y “Analyze phenotypic patterns and media moderation” (carril *External AI Service*) se ejecuten en paralelo, pero que la decisión final (“Incompatible kinship” vs. “Compatible information”) dependa de fusionar ambos resultados antes de bifurcarse hacia “Generate explicit biological risk alert” o “Calculate overall biological affinity percentage”, como se observa en la Figura 36, revela visualmente el mismo principio observado en el diagrama de secuencia CU-13 (Figura 23): el veredicto de riesgo determinista (parentesco) y el análisis probabilístico de la IA operan de forma independiente, pero la alerta de riesgo biológico tiene prioridad sobre el cálculo de afinidad cuando ambos resultados entran en conflicto.

1.10 Diagrama de Clases

1.10.1 Usuarios y Roles

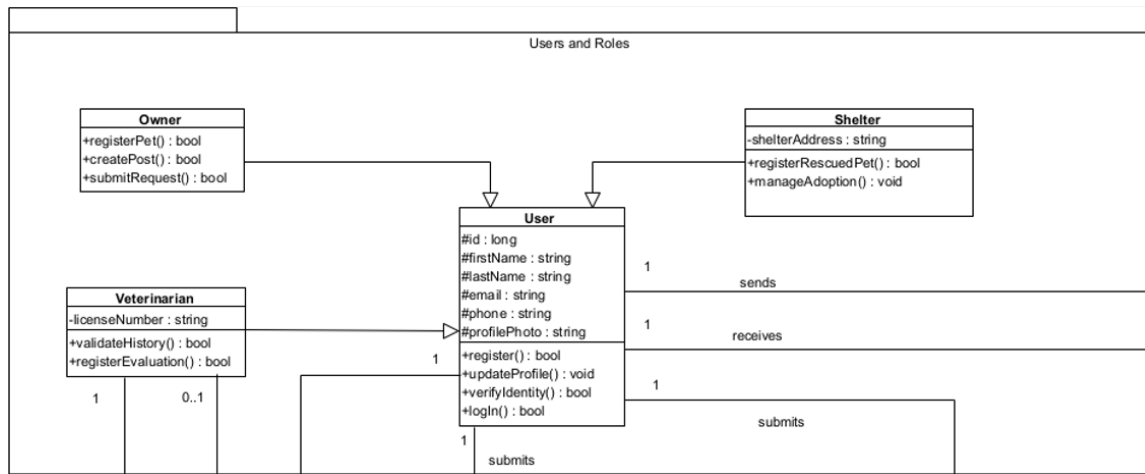


Figura 37: Diagrama de clases – Usuarios y Roles.

Que “Shelter”, “Owner” y “Veterinarian” hereden todos de una única clase base “User” (en lugar de modelarse como entidades independientes con su propia tabla de *login*), como se observa en la Figura 37, revela que el sistema trata la identidad y autenticación como algo unificado y transversal, mientras que el comportamiento específico de cada actor (*registerRescuedPet()*, *validateHistory()*, etc.) se resuelve por especialización. La ausencia deliberada de una clase “Administrator” (y de un mecanismo dinámico de roles tipo *Role/UserRole*) refleja que MundiPets opera como una plataforma entre pares sin un rol administrativo central: el tipo de usuario — y por tanto sus permisos — queda resuelto enteramente por la propia jerarquía de herencia, sin necesitar una capa de gestión de roles independiente que ningún requisito funcional respalda.

Corrección aplicada (diagnóstico de cobertura, Sección 2): las clases *Administrator*, *Role* y *UserRole* se eliminaron de esta partición del diagrama de clases por constituir una cadena rota — ningún RF ni CU las respaldaba—. El control de acceso por tipo de usuario queda completamente cubierto por la herencia *Shelter/Owner/Veterinarian* → *User*, ya utilizada de forma consistente en el campo “Actor” de los 27 requisitos funcionales del ERS.

1.10.2 Mascotas y Salud

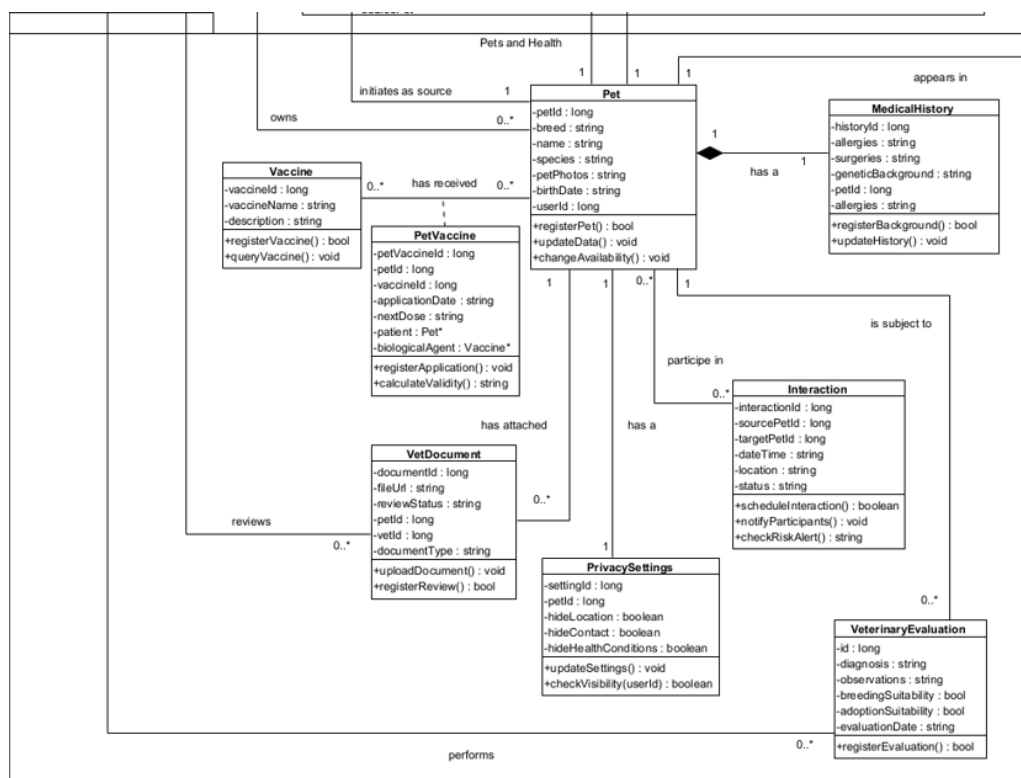


Figura 38: Diagrama de clases – Mascotas y Salud.

Que “MedicalHistory” se relacione con “Pet” mediante una composición (rombo relleno, “has a” 1 a 1) y no una simple asociación, como se aprecia en la Figura 38, revela que el historial médico no tiene existencia independiente de la mascota: si se elimina el perfil de la mascota, su historial desaparece con él, a diferencia de “VeterinaryEvaluation” o “VetDocument”, que se asocian de forma más débil (0..*) y podrían sobrevivir como registros históricos aun si la mascota deja de estar activa. Por otro lado, que “PetVaccine” sea una clase de asociación entre “Pet” y “Vaccine” (con atributos propios como *nextDose* y *biologicalAgent*) en lugar de una relación directa, muestra que el sistema necesita rastrear cada aplicación individual de una vacuna (con su propia fecha y próxima dosis) y no solo si la mascota “tiene” cierta vacuna en abstracto. Que “Interaction” reutilice el mismo patrón estructural que “BreedingRequest” (dos referencias a “Pet”, origen y destino, más un método *checkRiskAlert()*) revela que el sistema reconoce que cualquier vínculo entre dos mascotas concretas, sea un encuentro de socialización o una solicitud de cruce, comparte el mismo riesgo biológico de fondo y debe validarse con la misma lógica, en lugar de duplicar esa regla de negocio en dos lugares distintos del modelo. Que “PrivacySettings” se relacione con “Pet” mediante composición 1 a 1 (“has a”), igual que “MedicalHistory”, muestra que la configuración de privacidad no es un ajuste global de la cuenta del usuario, sino un dato que pertenece y viaja con cada mascota individual: dos mascotas del mismo propietario pueden tener reglas de visibilidad distintas entre sí.

1.10.3 Comunicación y Publicaciones

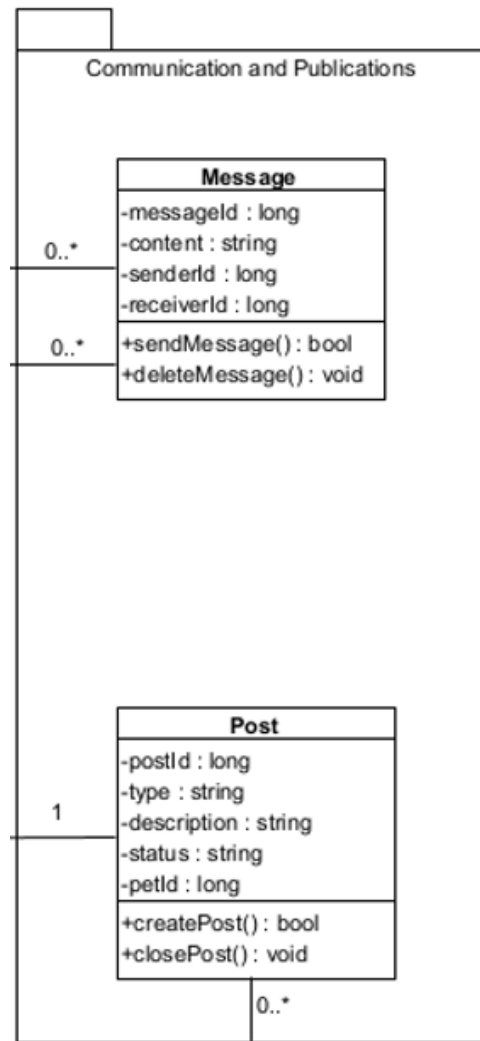


Figura 39: Diagrama de clases – Comunicación y Publicaciones.

Que “Message” siga sin ninguna relación directa con “Post”, ni siquiera ahora que existe “Interaction” para modelar encuentros presenciales (partición Mascotas y Salud), como se observa en la Figura 39, refuerza la misma lectura de diseño ya identificada: la mensajería es un canal transversal entre usuarios, desacoplado tanto del contexto de una publicación como del registro formal de un encuentro coordinado. Esto evita que “Message” termine sobrecargado con responsabilidades que no le corresponden (agendar, registrar ubicación, marcar estado de un encuentro), las cuales fueron correctamente delegadas a una clase propia (“Interaction”) en lugar de resolverse con campos adicionales dentro del propio mensaje.

1.10.4 Solicitudes y Evaluaciones

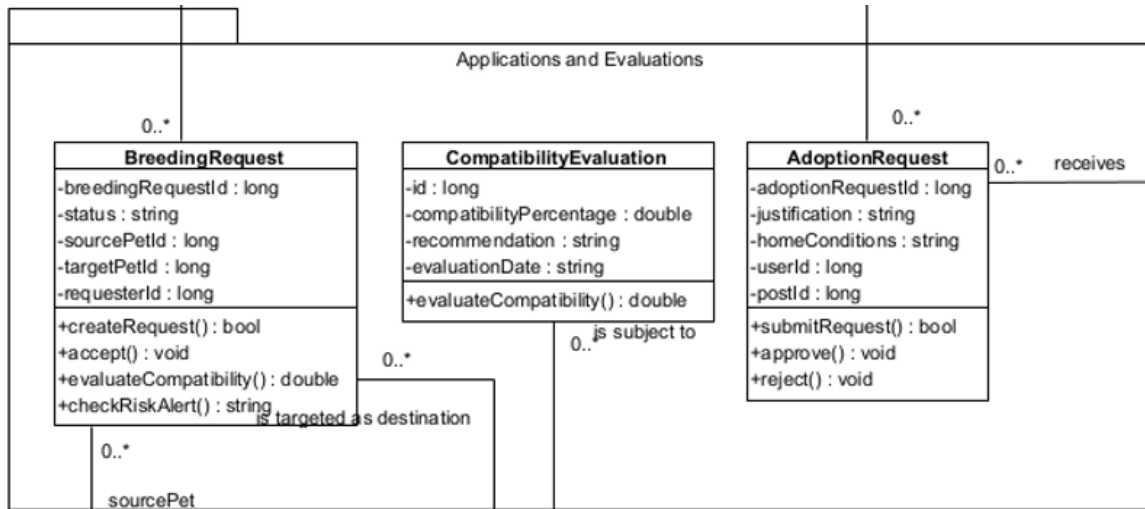


Figura 40: Diagrama de clases – Solicitudes y Evaluaciones.

Que *checkRiskAlert()* se agregue como método propio de “BreedingRequest” (y no como una clase de evaluación independiente, a diferencia de “CompatibilityEvaluation”), como se ve en la Figura 40, revela que la alerta de riesgo sanitario o físico se trató deliberadamente como una validación transitoria calculada al momento de crear la solicitud, no como un resultado que deba persistirse y auditarse después: mientras “CompatibilityEvaluation” sí se guarda como registro propio (porque su recomendación debe poder consultarse más adelante), la alerta de riesgo solo necesita bloquear o advertir en el instante de la confirmación, y desaparece una vez tomada la decisión.

1.10.5 Vista Refinada General

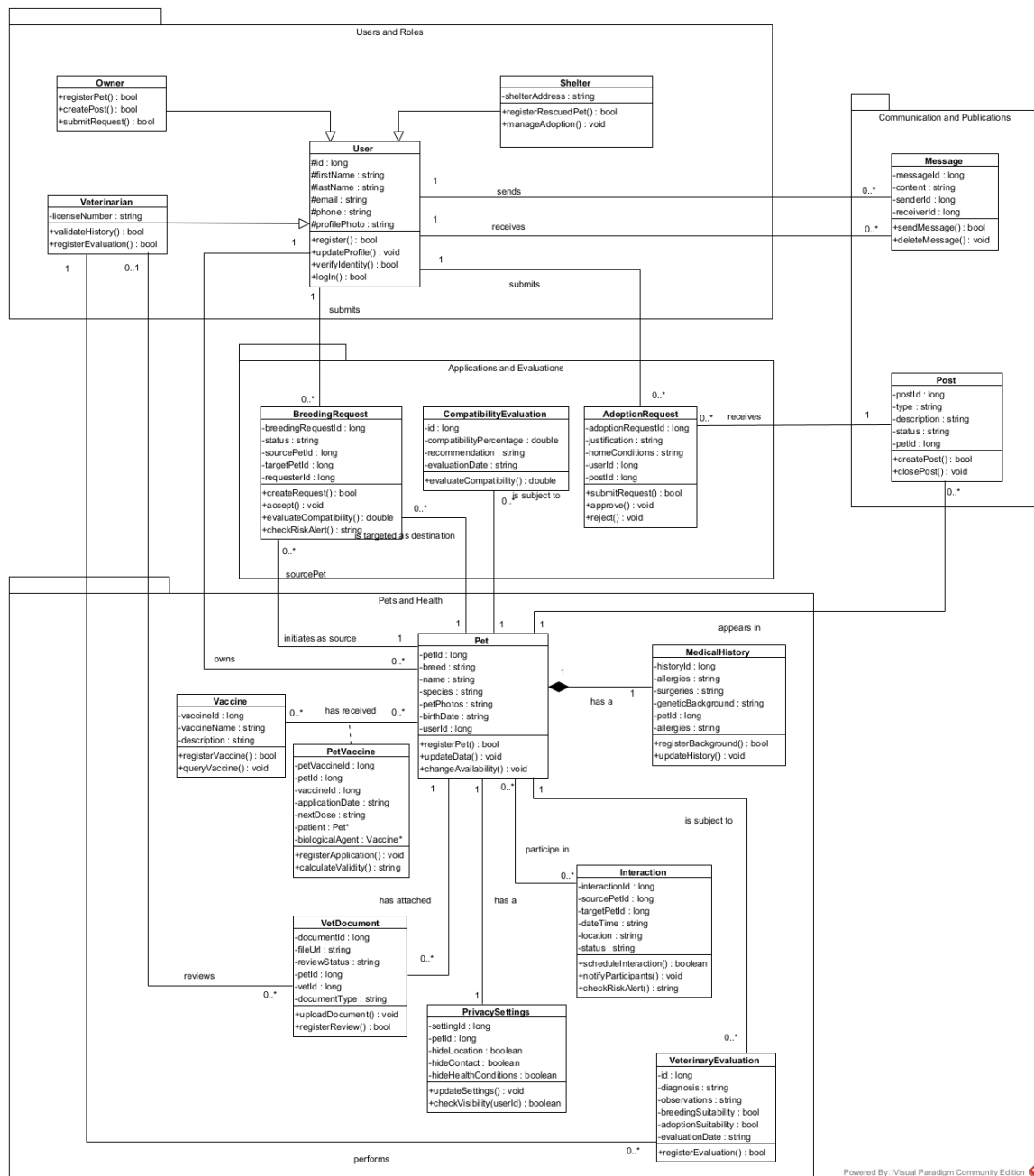


Figura 41: Diagrama de clases – Vista refinada general (integración de las cuatro particiones).

Que “Pet” sea la clase con más conexiones entrantes de todo el diagrama (origen de “AdoptionRequest”, “BreedingRequest” vía *sourcePet* / “is targeted as destination”, “MedicalHistory”, “PetVaccine”, “VetDocument”, “VeterinaryEvaluation” y “Post”), como se observa en la Figura 41, confirma visualmente lo que ya el diagrama de Contexto (Figura 2) y la matriz de trazabilidad (Sección 2) venían señalando: la mascota es el agregado central del dominio de MundiPets, y prácticamente ningún proceso (adopción, cruce, salud, comunicación) tiene sentido sin anclarse primero a un “Pet” concreto. Esto también explica por qué la Sección ?? (trazabilidad) probablemente encuentre más filas asociadas a RF relacionados con “Pet” que con cualquier otra entidad, y por qué un defecto en el modelo de “Pet” tendría el radio de impacto más amplio de todo el sistema (relevante para M5, modificabilidad).

Corrección aplicada (diagnóstico de cobertura, Sección 2): la clase *VeterinaryAppointment* (agendar/cancelar citas) se elimina de esta partición por constituir una cadena rota que además contradice dos exclusiones de alcance declaradas explícitamente en el ERS: MundiPets no reemplaza la consulta veterinaria presencial ni ofrece una

agenda médica completa de clínicas. RF-10 (recordatorios de controles preventivos) cubre únicamente el aviso pasivo de vacunas y controles próximos, no la reserva activa de un turno dentro del sistema, por lo que ambas funciones no deben confundirse.

ewpage

2 Gestión y Trazabilidad

2.1 Línea base y control de cambios

Durante la Práctica Experimental 4 (PE4), el docente conformó grupos de inspección ajenos a los equipos del PFC. Como resultado, el ERS de MundiPets fue inspeccionado de manera independiente por cuatro subgrupos, cada uno integrado por un miembro del equipo del PFC junto con compañeros externos: Fuertes–Contreras–Viteri, Cedeño Ávila–Córdova–Mendoza, Morales–Sánchez–Cedeño y Gutiérrez–Nieves. Cada subgrupo tramitó tres RFC ante un Change Control Board (CCB) simulado, sobre su propia copia de trabajo del ERS. En total se generaron 12 formularios RFC, de los cuales tres correspondían al mismo hallazgo detectado de forma independiente por tres subgrupos distintos, quedando 10 solicitudes de cambio únicas. La Tabla 1 consolida estas 10 RFC, su decisión del CCB y su justificación.

Tabla 1: Registro de RFC tramitados en la PE4 sobre el ERS de MundiPets.

RFC	Requisito afectado	Decisión	Justificación	Fecha
RFC-A	RF-25, RNF-16	Aprobado / Aprobado con condiciones (3 subgrupos)	RNF-16 (base legal LOPDP) exige ocultar ubicación/contacto por defecto; RF-25 los exponía	08–09/08/2026
RFC-B	RD-06, RL-04	Aprobado	El propio RD-06 reconoce que ningún RF implementa el derecho de eliminación (Art. 15 LOPDP)	09/08/2026
RFC-C	RF-10, RD-08	Aprobado	RD-08 exige canal de respaldo ante caída de WhatsApp, pero RF-10 no lo formaliza	09/08/2026
RFC-D	RNF-09, RD-08	Aprobado con condiciones	Comprometer un proveedor de pago (WhatsApp) sin presupuesto aprobado es inviable	09/08/2026
RFC-E	RNF-02	Aprobado	El MVP local no permite verificar disponibilidad mensual del 99 %	09/08/2026
RFC-F	RF-07, RNF-15	Aprobado con condiciones	El ERS reconoce el conflicto “comunicación inadecuada” sin RF que lo resuelva	08/08/2026
RFC-G	RNF-03	Aprobado con condiciones	El umbral de 2s no era verificable con la infraestructura disponible	08/08/2026
RFC-H	RF-07, RNF-15	Aprobado con condiciones	RNF-15 exige explicabilidad de una moderación que ningún RF especifica	09/08/2026
RFC-I	RF-06, RNF-06, RNF-13	Aprobado	RNF-06/RNF-13 califican una salida que RF-06 no declaraba producir	09/08/2026
RFC-J	RF-01, RF-27, RL-01	Aprobado	El consentimiento debe ser revocable en cualquier momento (Art. 7 y 8 LOPDP)	09/08/2026

Ningún CCB de los cuatro subgrupos de la PE4 rechazó una RFC: las 10 solicitudes únicas fueron aprobadas o aprobadas con condiciones sobre la copia de trabajo de cada subgrupo. Sin embargo, ninguna de esas aprobaciones se tradujo automáticamente en un cambio sobre el repositorio único del PFC, ya que cada subgrupo trabajó de forma independiente sobre su propia copia del documento. Por ello, como parte del cierre de la PE5, el equipo del PFC convocó un nuevo CCB para decidir formalmente sobre las 10 RFC frente a la línea base real de MundiPets. De esas 10: una ya estaba resuelta en el ERS vigente (RFC-A); cinco se aplicaron directamente (RFC-C, RFC-E, RFC-F, RFC-I, y la corrección de alcance derivada de RFC-G); una se resolvió reacotando un requisito no funcional ya existente, sin crear un requisito funcional nuevo (RFC-H); y tres fueron rechazadas con justificación registrada, por preferirse una alternativa (RFC-D) o por no contar con evidencia de entrevista que sustentara un requisito funcional nuevo y verificable (RFC-B, RFC-J). La Tabla 2 documenta la decisión final de cada una; ninguna RFC permanece sin resolver al cierre de la PE5.

Tabla 2: Estado de incorporación en la línea base vigente (ERS v4.0).

RFC	Origen (subgrupo PE4)	Decisión final del CCB (cierre PE5)
RFC-A	Fuertes, Morales, Gutiérrez/Nieves	Resuelto: ya estaba correcto en el ERS
RFC-B	Mendoza	Rechazado, con justificación: crear el RF sin evidencia de entrevista lo volvería no verificable. RD-06 se mantiene como restricción reconocida
RFC-C	Mendoza	Aplicado: RF-10 corregido
RFC-D	Morales	Rechazado: se prefirió RFC-C como solución al mismo problema
RFC-E	Mendoza	Aplicado: RNF-02 corregido
RFC-F	Fuertes	Aplicado: RF-07 corregido
RFC-G	Fuertes	Rechazada la propuesta original (relajar el umbral); resuelto por vía alternativa: RNF-03 acotado a producción, igual que RNF-02
RFC-H	Morales	Resuelto: RNF-15 reacotado a RF-07, sin crear RF nuevo
RFC-I	Gutiérrez/Nieves	Aplicado: RF-06 corregido
RFC-J	Gutiérrez/Nieves	Rechazado, con justificación: requeriría un RF con efectos en cascada no verificable sin evidencia. RL-01 queda con brecha reconocida, no con RFC abierta

A la fecha de cierre de esta PE5, el equipo aún no ha establecido una línea base formal sobre el repositorio del PFC (<https://github.com/andymendozap03/MundiPets-PFC-IngReq>). Para declarar la línea base final, corresponde ejecutar:

```
git tag -a baseline-vX.X -m "Linea base final ERS MundiPets - PE5"
git push origin baseline-vX.X
```

y completar en esta sección el número de versión, la fecha y el *hash* del commit sobre el que quedó anclado el tag, de modo que la línea base sea auditable conforme exige el criterio C2 de la rúbrica.

2.2 Matriz de trazabilidad final

La Tabla 3 consolida la trazabilidad end-to-end de los 43 elementos vigentes del ERS (27 RF y 16 RNF), siguiendo la cadena mínima exigida para el PFC: Fuente/Stakeholder, RF/RNF, CU, Clase UML, Proceso DFD, Estado, Criterio BDD, Caso de Prueba conceptual e Historia del backlog, más una columna de trazabilidad horizontal (relación entre requisitos del mismo nivel) solicitada adicionalmente por el docente. El Proceso DFD corresponde a los seis procesos del Diagrama de Flujo de Datos Nivel 1 (P1 a P6, Figura 7), que descompone el proceso único “0. MundiPets” del Diagrama de Contexto (Figura 2). La columna Fuente identifica al participante de origen mediante su código de evidencia (PROP-XX para propietarios, VET-XX para veterinarias, OBS-01 para la observación de campo y Encuesta para el cuestionario digital agregado), conforme lo solicitó el docente en lugar del identificador EV-XX. Un enlace se marca únicamente cuando existe evidencia textual verificable en el ERS o en los modelos de la Sección 4; los eslabones sin relación real se marcan con “—”.

Tabla 3: Matriz de Trazabilidad Final del ERS de MundiPets.

RF/RNF	Fuente	Horizontal	CU	Clase UML	DFD	Estado	BDD	CP	Hist.	Estado de la traza
RF-01	PROP-02, VET-01, VET-03, OBS-01	RF-12	CU-01	Pet	P1	Perfil Mascota	CA-01	CP-01	HU-01	Completa
RF-02	PROP-01, VET-02	RF-14	CU-02	MedicalHistory, Pet	P2	—	CA-02	CP-02	HU-02	Completa
RF-03	VET-02, PROP-03, OBS-01	RF-11	CU-10	Pet, MedicalHistory	P2	—	CA-03	CP-03	HU-03	Completa
RF-04	Encuesta, OBS-01	—	CU-11	Post, Pet	P3	—	—	CP-04	—	Completa (Should)
RF-05	PROP-01, PROP-02, VET-01, OBS-01	RF-13	CU-04	Post	P3	Perfil Mascota	CA-04	CP-05	HU-04	Completa
RF-06	VET-02, VET-03, PROP-03, OBS-01	RNF-13, RF-02, RF-16	CU-13	CompatibilityEvaluation, Pet	P4	—	CA-05	CP-06	HU-05	Completa
RF-07	PROP-02, Encuesta, OBS-01	RNF-15	CU-08	Message	P5	—	—	CP-07	—	Completa (Could)
RF-08	PROP-01, VET-01, VET-03	RF-27	CU-05	BreedingRequest	P3	Solicitudes	CA-06	CP-08	HU-06	Completa
RF-09	VET-01, VET-03, Encuesta	RF-26, RF-19	CU-09	VeterinaryEvaluation, MedicalHistory	P2	Verif. Documentos	CA-07	CP-09	HU-07	Completa
RF-10	PROP-02, VET-02, PROP-03	RD-08	CU-12	Pet, PetVaccine	P5	—	—	CP-10	—	Completa (Should)
RF-11	PROP-02, VET-03, PROP-03, Encuesta, OBS-01	RF-03, RNF-01, RNF-16	CU-06	PrivacySettings	P1	—	CA-08	CP-11	HU-08	Completa
RF-12	PROP-01, VET-01, Encuesta, OBS-01	RF-01	(Onboarding)	User	P1	—	CA-09	CP-12	HU-09	Completa
RF-13	PROP-01, PROP-02, Encuesta, OBS-01	RF-05	CU-04	Post	P3	—	CA-10	CP-13	HU-10	Completa
RF-14	VET-01, VET-03, Encuesta	RF-02, RF-24	CU-03	PetVaccine, VetDocument	P2	Verif. Documentos	CA-11	CP-14	HU-11	Completa
RF-15	PROP-01, PROP-02, VET-02, PROP-03	RF-27	CU-07	AdoptionRequest, BreedingRequest	P3	Solicitudes	—	CP-15	—	Completa (Should)
RF-16	PROP-01, VET-01, VET-03, Encuesta	RF-06	CU-02	Pet	P2	—	CA-12	CP-16	HU-12	Completa
RF-17	PROP-01, VET-01, Encuesta, OBS-01	RNF-06, RNF-07	(Publicaciones)	Post	P6	—	—	CP-17	—	Completa (Should)
RF-18	PROP-06, VET-05	RF-05	CU-04	Post	P3	Perfil Mascota	CA-13	CP-18	HU-13	Completa
RF-19	PROP-06	RF-09	CU-09	VetDocument, VeterinaryEvaluation	P2	Verif. Documentos	—	CP-19	—	Completa (Should)
RF-20	PROP-08	—	CU-08	Message, Interaction	P5	—	—	CP-20	—	Completa (Could)
RF-21	VET-05	—	CU-01	Pet	P1	—	—	CP-21	—	Completa (Should)
RF-22	PROP-05, VET-05	—	CU-07	AdoptionRequest	P3	Solicitudes	CA-14	CP-22	HU-14	Completa
RF-23	PROP-04, PROP-07, VET-04, PROP-11	—	CU-06	Interaction, BreedingRequest	P4	—	CA-15	CP-23	HU-15	Completa
RF-24	VET-04, VET-05	RF-14	CU-03	PetVaccine	P2	—	—	CP-24	—	Completa (Should)
RF-25	PROP-07, VET-05	RNF-16	CU-04	Post	P3	Perfil Mascota	—	CP-25	—	Completa (Could)
RF-26	VET-07	RF-09	CU-09	VeterinaryEvaluation	P2	Verif. Documentos	—	CP-26	—	Completa (Should)
RF-27	PROP-12	RF-08, RF-15	CU-05	BreedingRequest	P3	Solicitudes	—	CP-27	—	Completa (Should)
RNF-01	VET-03, PROP-03, Encuesta, OBS-01	RF-11, RNF-16, RD-04, RNF-05	CU-06	—	P1	—	—	CP-28	—	Completa (transversal)
RNF-02	PROP-01, PROP-02, VET-01, VET-02, VET-03, PROP-03, Encuesta	RNF-03	(Transversal)	—	Transversal	—	—	CP-29	—	Completa (transversal)
RNF-03	PROP-01, PROP-02, VET-01, VET-02, VET-03, PROP-03, Encuesta	RNF-02	CU-11	—	Transversal	—	—	CP-30	—	Completa (transversal)
RNF-04	PROP-01, PROP-02, VET-01, VET-02, VET-03, PROP-03, Encuesta, OBS-01	—	(UI)	—	Transversal	—	—	CP-31	—	Completa (transversal)
RNF-05	VET-01, VET-03, Encuesta	RD-04, RNF-01	CU-02	—	P2	—	—	CP-32	—	Completa (transversal)
RNF-06	PROP-01, VET-01, VET-03, Encuesta	RF-06, RNF-13	CU-13	—	P4	—	—	CP-33	—	Completa (transversal)
RNF-07	PROP-01, VET-01, VET-03, Encuesta	RNF-06	(Media)	—	P6	—	—	CP-34	—	Completa (transversal)
RNF-08	PROP-01, PROP-02, VET-01, VET-02, VET-03, PROP-03, Encuesta	—	CU-01	—	P1	—	—	CP-35	—	Completa (transversal)
RNF-09	PROP-04, PROP-11	RD-08, RF-10	CU-12	—	P5	—	—	CP-36	—	Completa (transversal)
RNF-10	Derivado del equipo	—	(Diseño)	—	Transversal	—	—	CP-37	—	Completa (transversal)
RNF-11	Encuesta, OBS-01	—	(UI)	—	Transversal	—	—	CP-38	—	Completa (transversal)
RNF-12	Encuesta, OBS-01	RNF-03	(Transversal)	—	Transversal	—	—	CP-39	—	Completa (transversal)
RNF-13	PROP-06, PROP-08, VET-04, VET-05	RF-06, RNF-06, RNF-14	CU-13	—	P4	—	—	CP-40	—	Completa (transversal)
RNF-14	PROP-06, PROP-08	RF-06, RNF-13	(Media)	—	P6	—	—	CP-41	—	Completa (transversal)
RNF-15	PROP-08, PROP-11	RF-06, RNF-13, RF-07	CU-08	—	P5	—	—	CP-42	—	Completa (transversal)
RNF-16	PROP-07, VET-05	RF-11, RNF-01, RD-02, RF-25	CU-06	—	P1	—	—	CP-43	—	Completa (transversal)

De los 43 elementos evaluados, el 100 % cuenta con fuente identificada, caso de uso o designación transversal, proceso DFD, y caso de prueba conceptual. Los 27 RF cuentan además con clase UML propia (100 %, tras incorporar *PrivacySettings* e *Interaction* durante el diagnóstico de cobertura de la Sección 2) e historia de usuario sincronizada (27/27, tras completar las 12 historias faltantes de prioridad Should/Could). El criterio de aceptación en formato BDD y el estado de máquina solo aplican a los elementos que lo requieren por diseño (15 RF de prioridad *Must* para BDD, y las entidades con ciclo de vida propio, *Pet*, *Post*, solicitudes y verificación de documentos, para Estado), conforme a la convención ya declarada en el ERS; su ausencia en el resto de las filas no constituye una traza incompleta. Con ello, la matriz cierra con el 100 % de sus elementos en estado “Completa”, sin huérfanos, cadenas rotas ni trazas parciales pendientes.

2.3 Diagnóstico de cobertura: huérfanos y cadenas rotas

Tabla 4: Diagnóstico de cobertura: huérfanos, cadenas rotas y trazas parciales.

Identificador	Patología	Causa	Acción tomada
Clases <i>Administrator</i> , <i>Role</i> , <i>UserRole</i>	Cadena rota	Se modeló un subsistema completo de administración y gestión de roles (Diagrama de clases, partición Usuarios y Roles) sin que ningún RF lo especifique; “administrador” solo aparece como ejemplo genérico en el glosario del ERS.	Resuelto. Clases eliminadas del diagrama de clases (partición Usuarios y Roles). Se corrigió además RD-02, que aún mencionaba “administradores” en su descripción y en las tablas resumen. El control de acceso por tipo de usuario queda cubierto por la herencia <i>Shelter/Owner/Veterinarian</i> → <i>User</i> .
Clase <i>VeterinaryAppointment</i>	Cadena rota	Se modeló la gestión de citas veterinarias (agendar/cancelar), en la partición Mascotas y Salud, sin que exista RF ni CU correspondiente; contradice además dos exclusiones de alcance ya declaradas en el ERS (“no reemplaza consulta veterinaria”, “no ofrece agenda médica completa”).	Resuelto. Eliminada de la partición Mascotas y Salud. RF-10 (recordatorios pasivos) queda diferenciado de la reserva activa de turnos, que sigue fuera de alcance.
RF-11 (Administrar privacidad)	Traza parcial	Tenía CU-06, HU-08 y CA-08, pero ninguna clase propia en el diagrama de clases; las secuencias de CU-06 mencionan <i>PrivacySettingsRepository</i> , nunca modelada.	Resuelto. Se agregó la clase <i>PrivacySettings</i> (composición 1:1 con <i>Pet</i>), con los atributos y métodos usados en las secuencias de CU-06/CU-10.
RF-20	Traza parcial (hallazgo adicional, no detectado en la primera revisión)	Tiene CU-	Resuelto.

Nota sobre huérfanos: no se agrega ninguna fila de este tipo porque los 43 elementos (27 RF + 16 RNF) cuentan con fuente (código de participante o derivación del equipo) y al menos un CU o designación transversal. Cero requisitos huérfanos detectados.

Diagnóstico global: de 43 elementos evaluados, se identificaron inicialmente 0 huérfanos, 2 cadenas rotas y 2 trazas parciales; en el proceso de corrección se detectó una tercera traza parcial (RF-20) no capturada en la revisión inicial. Las 5 patologías identificadas quedaron resueltas mediante 2 eliminaciones de clases sin respaldo (*Administrator/Role/UserRole*, *VeterinaryAppointment*) y 2 incorporaciones al diagrama de clases (*PrivacySettings*, *Interaction*) más un método compartido (*checkRiskAlert()*). El diagnóstico de cobertura del ERS de MundiPets cierra sin patologías pendientes.

2.4 Sincronización entre el product backlog y el ERS

Herramienta de tablero declarada: Jira (proyecto Kanban “MundiPets”, clave KAN), con 5 Épicas, 27 Historias y 25 Subtareas.

Tabla 5: Verificación de sincronización entre el product backlog y el ERS.

Verificación	Total	Cumple	%
RF con al menos una historia de usuario asociada	27	27	100 %
Historias del backlog con un RF de origen identificado	27	27	100 %

La verificación se realizó en ambas direcciones, conforme exige la guía: en dirección RF → Historia, los 27 RF de prioridad Must, Should y Could del ERS (RF-01 a RF-27) tienen exactamente una tarjeta en Jira que los referencia explícitamente en el campo “RF asociado”, ningún RF quedó sin cobertura y ninguno tiene más de una historia duplicada; en dirección Historia → RF, las 27 tarjetas de tipo Historia apuntan cada una a un RF real y existente en el ERS, sin tarjetas huérfanas del lado del backlog. Cada tarjeta incluye además, en su descripción, el número de HU (coincide con las 27 historias HU-01 a HU-27 redactadas para este informe), la prioridad MoSCoW y la fuente de evidencia, replicando fielmente el contenido del ERS dentro del tablero.

Diagnóstico: sincronización del 100 % en ambas direcciones. No se detectaron RF sin historia, historias sin RF de origen, ni duplicados. Este resultado es consistente con haber completado las 12 historias faltantes (HU-16 a HU-27) antes de construir el tablero, lo que evitó que Jira heredara el vacío de cobertura que existía inicialmente solo en el documento del ERS.

Referencias

- [1] O. C. Z. Gotel y A. C. W. Finkelstein, “An analysis of the requirements traceability problem,” *Proceedings of IEEE International Conference on Requirements Engineering*, págs. 94-101, 1994. DOI: 10.1109/ICRE.1994.292398
- [2] J. Mucha, A. Kaufmann y D. Riehle, “A systematic literature review of pre-requirements specification traceability,” *Requirements Engineering*, vol. 29, n.º 2, págs. 119-141, 2024. DOI: 10.1007/s00766-023-00412-z